```
> SECURE ENCLAVE HANDSHAKE ..... [OK]
> LOCALPOLICY SIGNATURE (SEP) ..... [VERIFIED]
> MOUNTING /dev/disk3s1 (APFS, sealed, SSV) ..... [RO]
> ARM64E POINTER AUTH ..... [ENABLED]
> LOADING artifact index [14 DOMAINS] ..... [OK]
> VERIFYING command corpus [392 ENTRIES] ..... [OK]
> ROSETTA 2 TRANSLATION CACHE ..... [PRESENT]
> OPERATOR CLEARANCE ..... [GRANTED]
```

MACOS THREAT HUNTING & FORENSICS

APPLE SILICON · ARTIFACT-INDEXED FIELD GUIDE

>> A C C E S S G R A N T E D <<

BUILD	v2.0 STAMP 2026.07.30 supersedes v1.0
TARGET	Apple silicon (M-series) · macOS 26 Tahoe · back-compatible 13+
SCOPE	native CLI · EndpointSecurity · osquery · YARA · offline parsers
ORDER	by artifact domain, not by kill-chain phase
READER	operator-level. no primers, no framing.
FORMAT	one command per box · copy-paste clean · annotation and tags outside

>> INDEX

[01] PLATFORM INTEGRITY & BOOT POLICY ..... PG 03
[02] PROCESS & EXECUTION ..... PG 05
[03] SIGNING, NOTARIZATION & TRANSLATION ..... PG 07
[04] PERSISTENCE ..... PG 09
[05] NETWORK ..... PG 12
[06] FILESYSTEM & METADATA ..... PG 14
[07] OBFUSCATION, SPOOFING & MACH-O ..... PG 16
[08] USER ACTIVITY, BROWSER & APP ARTIFACTS ..... PG 18
[09] LOGS & UNIFIED LOGGING ..... PG 21
[10] VOLATILE CAPTURE & EVIDENCE HANDLING ..... PG 23
[11] OSQUERY ..... PG 25
[12] YARA & DETECTION ENGINEERING ..... PG 28
[13] ATT&CK MAPPING (macOS MATRIX) ..... PG 31
[14] QUICK TRIAGE CHECKLIST ..... PG 32


```
> run --section 01 
```

>> START HERE

01 Run section 01 and paste the output into your notes. If the LocalPolicy is not Full, or SIP is off, or the SSV seal is broken -- stop reading and image the disk.

02 Take the volatile capture in section 10 phase 1 before you disturb anything. The unified log is a ring buffer and your own hunting is pushing it out of scope.

03 Work section 14 for a fast pass, then drop into whichever domain the leads point at.

NO WARRANTY · COMMANDS ALTER STATE · READ BEFORE YOU RUN · VERIFY AGAINST YOUR OWN BUILD

HOW TO READ THIS GUIDE

Every entry is one command in its own box with the annotation directly beneath it. Boxes contain the command and nothing else -- no prompt glyphs, no labels, no decoration -- so selecting a box gives you something you can paste straight into a shell. Everything else, including elevation and version markers, sits outside the box on the annotation line.

NOTATION -- ALL OF IT OUTSIDE THE COMMAND BOXES

ROOT	Needs elevation. The command itself already carries sudo where required.
13+ / 26+	Minimum macOS version. Older estates need the fallback named in the annotation.
<...>	Placeholder you must substitute. Never paste one of these unedited.
!!	Operator callout: an inference, a trap, or a false-positive class.
↳	Annotation: what the output means and what makes it interesting.

Section 01 comes first for a reason. On Apple silicon the trust anchor is a Secure Enclave-signed LocalPolicy, and if that policy has been downgraded then every command in sections 02 through 14 is reporting from a platform that can lie to you. Run 01, write the output into your notes, and only then continue.

>> WHAT CHANGED FROM v1.0

DELTA

Silicon-native	Intel-era assumptions removed. arm64e, LocalPolicy, Rosetta 2, kmutil.
New S01	Platform integrity & boot policy. bputil, SSV seal, policy override surface.
New S03	Signing merged with translation. The Rosetta AOT cache is now a first-class artifact.
New S08	User activity, browser & app artifacts. Biome, Tahoe App.MenuItem, credential stores.
New S10	Volatile capture & evidence handling. Order of volatility, hashing, chain of custody.
New S13	ATT&CK macOS mapping as a coverage worksheet.
Expanded S12	YARA section now carries full rule bodies rather than module names alone.
DTrace demoted	eslogger is the default. DTrace costs a SIP change you should not pay.

!! Two clocks are running against this document. Rosetta 2 gets its last full release in macOS 27 Golden Gate and is cut to a gaming-only subset in macOS 28 -- so the AOT cache technique in section 03 has an expiry date, and x86-64 tradecraft expires with it. Re-verify this guide at each major release.

[01] PLATFORM INTEGRITY & BOOT POLICY

Establish what the silicon will still vouch for before you trust a single result below.

Every finding in sections 02-14 is only as trustworthy as the boot chain that produced it. On Apple silicon the trust anchor is the Secure Enclave-signed LocalPolicy, not a firmware password or an NVRAM variable. Run this section first and record the output verbatim.

>> SECURE BOOT / LOCALPOLICY

```
sudo bputil -d
```

↳ **[ROOT]** Dump the full LocalPolicy for the current boot volume group: security mode, signed system volume state, kext approval, MDM policy pinning, and the SEP-signed nonce. This is the single most important command on the page.

```
sudo bputil -d | grep -E 'Security Mode|Signed System Volume|Kernel CTRR|3rd Party Kexts'
```

↳ **[ROOT]** Four-line verdict. Expect Security Mode: Full, SSV enabled, CTRR enabled, 3rd Party Kexts disabled. Anything else is a deliberate downgrade someone authenticated to perform.

```
csrutil status ; csrutil authenticated-root status
```

↳ SIP state plus the authenticated-root seal. Authenticated-root disabled means the Signed System Volume seal was intentionally broken -- the system volume can no longer be trusted to be Apple's.

```
fdesetup status ; fdesetup list
```

↳ FileVault state and enabled users. On silicon the volume is always encrypted at rest; FileVault controls whether a password is required to unlock the key.

```
sudo nvram -p | grep -iE 'boot-args|policy|recovery'
```

↳ **[ROOT]** Custom boot-args should be empty on a production host. Any value here survived a Reduced-Security authentication.

```
system_profiler SPiBridgeDataType SPHardwareDataType
```

↳ Chip family, Secure Enclave / iBridge model, hardware UUID, provisioning UDID. Records which silicon generation you are actually standing on.

```
ioreg -l | grep -iE 'IOPlatformSerialNumber|board-id|target-type'
```

↳ Board identity from the device tree. Cross-check against the serial the fleet inventory claims for this host.

!! Security Mode: Permissive means iBoot will accept boot objects signed by the Secure Enclave with the LocalPolicy key -- custom kernels included. On a production endpoint this is a full compromise of the platform trust model. Stop, image the disk, escalate.

LOCALPOLICY SECURITY MODES -- WHAT EACH ONE CONCEDES

Full	Only the OS that was current at install time boots. iOS-equivalent posture.
Reduced	Global signatures trusted; older macOS boots; 3rd-party kexts permitted.
Permissive	SEP-signed custom boot objects accepted. Required for custom XNU kernels.

>> SYSTEM VOLUME SEAL & SNAPSHOTS

```
diskutil apfs list | grep -A4 'Snapshot'
```

↳ APFS snapshot inventory across containers. The sealed system snapshot is your known-good baseline; user data snapshots are your pre-incident diff source.

```
tmutil listlocalsnapshots /
```

↳ Local APFS snapshots on the data volume. Each is a mountable point-in-time you can diff against the live filesystem.

```
sudo mount_apfs -o ro -s com.apple.TimeMachine.<stamp>.local / /tmp/snap
```

↳ **[ROOT]** Mount a named local snapshot read-only. Diff /tmp/snap against live paths to prove what changed since that timestamp without trusting any on-host timestamp.

```
sudo /usr/libexec/seputil --status 2>/dev/null || echo 'seputil unavailable'
```

↳ **[ROOT]** Secure Enclave utility status where present. Availability varies by build -- treat absence as inconclusive, not as a negative result.

>> POLICY OVERRIDE SURFACE

```
sudo profiles list -all -verbose
```

↳ **[ROOT]** Every configuration profile including device-scoped, with payload UUIDs. Profiles can silently grant PPPC/TCC, install trusted roots, and disable security prompts.

```
ls -la /var/db/ConfigurationProfiles/Store/ /Library/Managed\ Preferences/
```

↳ On-disk profile store and MDM-enforced preference domains. Compare against what your MDM believes it has deployed -- divergence is the finding.

[01] > PLATFORM INTEGRITY & BOOT POLICY

```
sudo security dump-trust-settings -d ; security find-certificate -a -p /Library/Keychains/System.keychain | grep -c
  CERTIFICATE
```

↳ **[ROOT]** Admin trust overrides and system keychain certificate count. An injected root CA makes all TLS inspection invisible to you.

```
spctl --status ; spctl --status --verbose ; sudo spctl --list | head -40
```

↳ **[ROOT]** Gatekeeper global state plus the assessment rule database. Explicit allow rules for non-Apple developer IDs are worth reading line by line.

```
sudo launchctl print-disabled system | grep -v 'disabled => false'
```

↳ **[ROOT]** Jobs disabled via override -- the standard way to mute a security agent without deleting it. Check for XProtect, MRT, and your own EDR labels.

!! Record SIP, LocalPolicy, Gatekeeper and log-redaction state at the top of your notes with a timestamp. If any are weakened, every subsequent on-host result is advisory only and the correct move is a disk image plus offline analysis.

[02] PROCESS & EXECUTION

What is running, what spawned it, what architecture it is running as.

>> ENUMERATION

```
ps -axo pid,ppid,uid,user,lstart,etime,%cpu,%mem,args
```

↳ Full process table with true start time and ancestry. lstart gives absolute wall-clock start rather than the relative form, which matters when you correlate against log windows.

```
ps -axo pid,ppid,args | awk '$2==1 && $1!=1'
```

↳ Processes reparented to launchd. Legitimate for daemons; suspicious for anything living in a user-writable path.

```
ps -axo pid,args,arch 2>/dev/null || ps -axo pid,args -o 'arch='
```

↳ Per-process architecture. On silicon, arm64 is native, arm64e is pointer-authenticated, x86_64 means the process is running under Rosetta 2 translation -- see section 03.

```
pgrep -fl <pattern>
```

↳ Match against full argv rather than just the truncated comm name. Malware frequently uses an innocuous comm with a revealing command line.

```
launchctl procinfo <pid>
```

↳ launchd's own view of a process: job label, domain, on-disk path, sandbox container, and entitlement flags. Frequently contradicts what ps reports.

```
launchctl print pid/<pid>
```

↳ Live job state, inherited environment, and spawn path. Read the environment block for DYLD_* injection variables.

```
lsappinfo list | grep -E 'bundlePath|ASN|launch reason'
```

↳ Registered GUI applications with their ASN, bundle path, and why they were launched. Reveals apps with no Dock presence.

```
lsof -p <pid>
```

↳ Open files, sockets, loaded dylibs, and cwd for a single process. Your fastest route from a suspicious PID to its on-disk footprint.

```
lsof -c <name> -nP
```

↳ Everything opened by every process matching a command name, numeric addresses only.

```
vmmap <pid> | grep -iE 'mapped file|__TEXT'
```

↳ Mapped images including injected or non-bundled dylibs. A dylib mapped from /tmp or a user directory into a signed process is the finding.

```
sample <pid> 5 -f /tmp/sample.txt ; head -60 /tmp/sample.txt
```

↳ Five-second stack sample. Shows the call paths a process is actually executing, which survives obfuscated argv.

```
sudo spindump <pid> 5 -file /tmp/spin.txt
```

↳ **[ROOT]** Heavier kernel-assisted sample including thread states and binary images with UUIDs. Use when sample output is too shallow.

```
ps -axo pid,args | grep -Ei 'base64|curl |osascript|-e |eval|/tmp/|\.helper'
```

↳ Argv-visible staging and living-off-the-land invocation. Current infostealer families leave exactly this pattern behind.

```
sudo lsof +c 0 -w / | awk '$9 ~ /^\/(tmp|Users\/Shared|private\/tmp)/'
```

↳ **[ROOT]** Every open handle rooted in a world-writable path, with full command names rather than truncated ones.

>> ENVIRONMENT & INJECTION

```
for p in $(pgrep -U $UID); do echo "-- $p"; launchctl procinfo $p 2>/dev/null | grep -iE 'DYLD|LD_|INSERT'; done
```

↳ Sweep every user process for DYLD injection variables. DYLD_INSERT_LIBRARIES is the macOS equivalent of LD_PRELOAD and is stripped by hardened runtime -- its presence means the target was not hardened.

```
sudo eslogger exec | jq -r 'select(.event.exec.env != null) | [.process.executable.path, (.event.exec.env|join(" "))] |
  @tsv' | grep -i dyld
```

↳ **[ROOT]** Live EndpointSecurity stream filtered to execs carrying DYLD_* in their environment. The closest thing to native EDR telemetry macOS offers.

```
codesign -d --entitlements :- --xml <path> 2>/dev/null | plutil -p - | grep -iE
  'disable-library-validation|get-task-allow|allow-dyld-environment-variables'
```

↳ Three entitlements that together make a signed app injection-ready. Present on a shipping production app, this is a supply-chain question.

>> EXECUTION HISTORY

```
sudo sqlite3 /var/db/SystemPolicyConfiguration/ExecPolicy 'select * from policy_scan_cache order by last_seen desc
  limit 40;'
```

↳ **[ROOT]** ExecPolicy scan cache -- first-execution records with CDHash and scan verdict. Survives deletion of the binary itself.

```
sudo sqlite3 /var/db/SystemPolicyConfiguration/ExecPolicy 'select responsible_file_identifier,
  datetime(timestamp+978307200,"unixepoch") from legacy_exec_history_v4 order by 2 desc limit 40;' 2>/dev/null
```

↳ **[ROOT]** Legacy exec history where the table exists on your build. Table names drift between releases -- run .tables first.

```
sudo sqlite3 /var/db/SystemPolicyConfiguration/ExecPolicy .tables
```

↳ **[ROOT]** Enumerate the actual schema on this build before writing queries against it. Never assume table names from a guide, including this one.

```
ls -laO /var/db/oah/*/*/ 2>/dev/null | head -40
```

↳ **[ROOT]** Rosetta 2 ahead-of-time cache. Each .aot file is durable evidence that a specific x86-64 binary executed on this host. See section 03 -- this is the highest-value execution artifact on Apple silicon.

```
sqlite3 ~/Library/Application\ Support/com.apple.spotlight/appIndex.db 'select * from app_index limit 20;' 2>/dev/null
```

↳ Spotlight application index. Schema varies; use as a corroborating source for app presence and first-seen time.

!! Absence of an execution record is never proof of non-execution. Corroborate across ExecPolicy, Rosetta AOT cache, unified log, FSEvents, and Biome (section 08) before you write a negative finding.

[03] SIGNING, NOTARIZATION & TRANSLATION

Signing posture on arm64e, and the Rosetta 2 evidence trail that outlives the payload.

>> SIGNATURE INSPECTION

codesign -dv --verbose=4 <path>

↳ Identifier, TeamIdentifier, CDHash, signing flags, and the full Authority chain. Your baseline signing question for any binary.

codesign -vvv --deep --strict <path>

↳ Validate the seal across every nested bundle. --deep is slow and misses some nesting cases; treat a pass as necessary but not sufficient.

codesign -d --entitlements :- --xml <path> | plutil -p -

↳ Dump the entitlements plist in readable form. Read it, do not skim it -- the interesting grants are usually three keys deep.

codesign -d -r- <path>

↳ Designated requirement string: the actual predicate the system evaluates when deciding whether a future version of this code is the same code.

codesign -dv <path> 2>&1 | grep -E 'Signature=adhoc|not signed|TeamIdentifier=not set'

↳ One-line triage for ad-hoc and unsigned binaries. Ad-hoc plus absent TeamIdentifier plus no quarantine bit is a strong lead.

codesign -dv --verbose=4 <path> 2>&1 | grep -E 'CDHash|flags='

↳ Extract the CDHash for threat-intel lookup and the flags field for runtime/library-validation posture.

spctl -a -vvv -t exec <path>

↳ Gatekeeper assessment for execution. Reports source=Notarized Developer ID, source=Unnotarized Developer ID, or rejection.

spctl -a -vvv -t install <dmg|pkg>

↳ Assessment for installers and disk images -- a different code path from exec assessment and frequently a different verdict.

stapler validate <path>

↳ Is a notarization ticket actually stapled to this bundle, or is the app relying on an online check that may never have happened?

syspolicy_check distribution <path> ; syspolicy_check notary-submission <path>

↳ Apple's own consolidated signing and notarization verdict. Clearer output than spctl and the closer match to what the OS will decide at launch.

>> ARM64E & POINTER AUTHENTICATION

file <bin> ; lipo -detailed_info <bin>

↳ Architecture inventory and per-slice offsets. On silicon expect arm64 or arm64e; an x86_64-only slice on a 2026 endpoint is a deliberate choice by the author.

otool -hv <bin> | grep -E 'cputype|cpusubtype|flags'

↳ Mach-O header. CPU_SUBTYPE_ARM64E indicates pointer authentication; PIE and hardened-runtime flags live in the same field.

otool -arch arm64e -l <bin> | grep -A3 LC_BUILD_VERSION

↳ Minimum OS and SDK the slice was built against. An SDK far older than the binary's file timestamps suggests a repackaged or backdated build.

Apple silicon enforces stricter code-signing on arm64/arm64e than on translated x86-64: unsigned arm64 code will not execute, while a self-signed x86-64 binary will run under Rosetta 2. This asymmetry is why sophisticated macOS malware has been shipping as x86-64 rather than native -- and it is precisely what makes the Rosetta cache below so valuable.

>> ROSETTA 2 (OAH) -- EXECUTION EVIDENCE THAT SURVIVES DELETION

sudo ls -laR /var/db/oah/ | head -60

↳ **[ROOT]** The AOT cache. Path is /var/db/oah/<install-UUID>/<binary-UUID>/<name>.aot. Owned by _oahd and SIP-protected, which is why attackers routinely fail to clean it.

sudo find /var/db/oah -name '*.aot' -exec stat -f '%SB %N' -t '%F %T' {} \; 2>/dev/null | sort

↳ **[ROOT]** Chronological list of every x86-64 binary first translated on this host. Birth time is the first-execution timestamp for that exact binary state.

sudo find /var/db/oah -name '*.aot' -newermt '2026-07-01' -exec shasum -a 256 {} \;

↳ **[ROOT]** Hash AOT files created after a pivot date. Two AOT files under different binary-UUID directories with identical hashes usually means only file metadata changed.

sudo find /var/db/oah -name '*.in_progress'

↳ **[ROOT]** Translation that started and never completed -- a crash, a kill, or an interrupted execution mid-heist.

```
log show --last 7d --info --predicate 'process == "oahd"'
```

↳ Rosetta daemon log: AOT lookup requests, cache hits, and translation start/finish. Binary paths are marked <private> unless you install an unredaction profile.

```
sudo strings -a /var/db/oah/*/*/*.aot | grep -iE 'DerivedData|/Users/|\.o$' | sort -u | head -40
```

↳ **[ROOT]** Developer path strings frequently survive in AOT files. In real DPRK casework these strings provided attribution when the original binary had been deleted.

!! Rosetta 2 receives its last full release in macOS 27 Golden Gate and is reduced to a gaming-only subset from macOS 28. Two consequences: x86-64 tradecraft has a hard expiry, and this artifact class disappears with it. Bank the technique now; plan for arm64-native malware being the default case.

```
sudo ls /var/db/oah/ | wc -l
```

↳ **[ROOT]** Should be one install-UUID directory. The cache is wiped and re-keyed on OS update, so multiple directories mean either a recent update or something odd.

>> XPROTECT & REMEDIATION HISTORY

```
sudo sqlite3 /var/protected/xprotect/XPdb 'select * from dates;' 2>/dev/null; ls -la /var/protected/xprotect/ 2>/dev/null
```

↳ **[ROOT]** XProtect Remediator scan bookkeeping. Confirms the scanner is actually running and when it last completed.

```
log show --last 30d --info --predicate 'subsystem == "com.apple.XProtectFramework.PluginAPI" OR process CONTAINS
  "XProtect"'
```

↳ XProtect and XProtect Remediator scan and detection results. A remediation event is proof malware was present and was silently cleaned -- a timeline anchor your EDR may have missed.

```
sudo eslogger xp_malware_detected xp_malware_remediated | jq .
```

↳ **[ROOT 13+]** Live EndpointSecurity stream for XProtect detection and remediation events. Present from macOS 13 onward.

```
ls -la /Library/Apple/System/Library/CoreServices/XProtect.bundle/Contents/Resources/ ; plutil -p
  /Library/Apple/System/Library/CoreServices/XProtect.bundle/Contents/Info.plist | grep -i version
```

↳ XProtect definition version. Stale definitions change how you weight a clean scan result.

```
ls -la ~/.Trash/ /Users/*/.Trash/ 2>/dev/null
```

↳ XProtect Remediator relocates detections to the user Trash. Files here with recent timestamps and executable bits are worth hashing before anyone empties it.

>> FLEET SWEEPS

```
find /Applications ~/Applications -maxdepth 3 -name '*.app' -exec codesign -vv {} \; 2>&1 | grep -v 'valid on disk'
```

↳ Every application that fails signature validation. Run it first on any host; the output is usually short and always interesting.

```
find /usr/local/bin /opt /tmp /var/tmp /Users/Shared -type f -perm -u+x -exec codesign -dv {} \; 2>&1 | grep -B1 -E
  'not signed|adhoc'
```

↳ Unsigned or ad-hoc executables in writable paths outside SIP protection.

```
system_profiler SPInstallHistoryDataType | grep -B2 -A4 'Source: '
```

↳ Package install history with dates and install source. Correlate against the quarantine ledger in section 06.

```
system_profiler SPApplicationsDataType | grep -B6 'Obtained from: Unknown'
```

↳ Applications with no identifiable signing origin.

```
sudo find / -type f -perm -u+x -not -path '/System/*' -not -path '/usr/*' -newermt '2026-07-01' -exec codesign -dv {}
  \; 2>&1 | grep -E 'adhoc|not signed'
```

↳ **[ROOT]** Time-bounded whole-disk sweep for recently written unsigned executables outside the sealed volume. Slow -- run it while you work the rest of the guide.

[04] PERSISTENCE

Every load point that survives a reboot, ordered by how often it is actually abused.

>> LAUNCHD -- AGENTS & DAEMONS

```
ls -la@ ~/Library/LaunchAgents /Library/LaunchAgents /Library/LaunchDaemons
```

↳ User and system job plists with extended attributes shown. The @ matters: a quarantine xattr on a LaunchAgent tells you it arrived by download.

```
launchctl list | awk '$1!="-"'
```

↳ Jobs currently loaded and holding a live PID, filtering out the inert majority.

```
launchctl print gui/$UID | less
```

↳ Complete user domain: jobs, endpoints, inherited environment, and paths. Long output, worth reading.

```
sudo launchctl print system | less
```

↳ **[ROOT]** Complete system domain. Same structure, higher privilege, fewer legitimate surprises.

```
sudo launchctl dumpstate > /tmp/launchd-state.txt ; wc -l /tmp/launchd-state.txt
```

↳ **[ROOT]** Full launchd state snapshot for offline diffing against a known-good host of the same build.

```
plutil -p <plist>
```

↳ Readable dump of a binary or XML plist without converting the file in place.

```
plutil -convert xml1 -o - <plist>
```

↳ Normalise a binary plist to XML on stdout for grep and diff. Leaves the original untouched, which matters for evidentiary integrity.

```
grep -rlE 'RunAtLoad|KeepAlive|StartInterval' /Library/Launch* ~/Library/LaunchAgents 2>/dev/null
```

↳ Auto-start and self-respawning jobs -- the subset that actually provides persistence.

```
for f in /Library/Launch*/*.plist ~/Library/LaunchAgents/*.plist; do echo "== $f"; plutil -p "$f" 2>/dev/null | grep
  -A4 -E 'ProgramArguments|Program'; done
```

↳ Program target for every job on the host. This is where you find the payload path.

```
for f in /Library/Launch*/*.plist; do l=$(plutil -extract Label raw "$f" 2>/dev/null); b=$(basename "$f" .plist); [
  "$l" != "$b" ] && echo "MISMATCH $b -> $l"; done
```

↳ Label versus filename mismatch across every system job. A classic tell, and this loop finds it without you reading a hundred plists.

```
sudo launchctl print-disabled system | grep -v 'false'
```

↳ **[ROOT]** Disabled and overridden jobs -- how a security agent gets masked without being deleted.

!! Known-current tradecraft: a LaunchDaemon labelled com.finder.helper pointing at a hidden dotfile payload such as ~/.helper, ~/.agent or ~/.mainhelper. Hunt the pattern -- hidden dotfile executable referenced by a plist -- not the specific name.

```
ls -la ~/.[a-z]* /Users/*/.[a-z]* 2>/dev/null | grep -vE
  'Trash|CFUserText|DS_Store|cache|zsh|bash|ssh|config|local|npm|git' | head -40
```

↳ Hidden dotfiles in home directories with the routine ones filtered out. Cross-reference anything executable against the ProgramArguments you just dumped.

>> BACKGROUND TASK MANAGEMENT (BTM)

```
sudo sfltool dumpbtm
```

↳ **[ROOT 13+]** The authoritative background-item inventory: agents, daemons, login items, and developer tools, with their developer identity and enabled state. Single best persistence command on modern macOS.

```
sudo sfltool dumpbtm | grep -E 'Items|UUID|Name|Developer Name|Disposition|Executable Path'
```

↳ **[ROOT 13+]** Condensed BTM view. Disposition encodes enabled/disabled and allowed/denied -- read it rather than assuming presence means active.

```
ls -la ~/Library/Application\ Support/com.apple.backgroundtaskmanagementagent/
```

↳ The BTM store on disk. Copy BackgroundItems-v*.btm for offline parsing when you cannot run sfltool.

```
osascript -e 'tell application "System Events" to get the name of every login item'
```

↳ Legacy login items, which still work and are still used.

```
defaults read com.apple.loginwindow 2>/dev/null
```

↳ LoginHook, LogoutHook, and auto-login artefacts. Deprecated for years, functional anyway.

```
ls -la /Library/PrivilegedHelperTools/ ; plutil -p /Library/LaunchDaemons/*.plist 2>/dev/null | grep -B4
  PrivilegedHelperTools
```

↳ SMJobBless privileged helpers -- a root-level persistence and privilege-escalation surface that most triage skips.

>> SCHEDULED EXECUTION

```
crontab -l 2>/dev/null ; sudo crontab -l -u root 2>/dev/null
```

↳ Per-user cron tables for the current user and root.

```
sudo ls -la /usr/lib/cron/tabs/ /etc/cron* 2>/dev/null
```

↳ **[ROOT]** On-disk crontabs and drop-in directories, including tables for users who never log in.

```
ls -la /etc/periodic/{daily,weekly,monthly}/ /etc/*.local 2>/dev/null
```

↳ Periodic script directories and the rc.local-style hooks. Rarely audited, trivially writable with root.

```
atq ; sudo atq ; ls -la /var/at/jobs/ 2>/dev/null
```

↳ Queued at(1) jobs for the current user and root.

```
cat /etc/zshenv /etc/zprofile ~/.zshrc ~/.zprofile ~/.zshenv ~/.bash_profile 2>/dev/null
```

↳ Shell rc persistence. /etc/zshenv is the dangerous one -- it fires for non-interactive shells too, so every script execution triggers it.

```
ls -la ~/Library/Workflows/Applications/Folder\ Actions/ ~/Library/Scripts/ 2>/dev/null
```

↳ Folder Actions attach a script to filesystem events on a directory; execution happens when the folder changes, with no launchd entry at all.

>> TCC / PPPC

```
sudo sqlite3 /Library/Application\ Support/com.apple.TCC/TCC.db 'select
  service,client,client_type,auth_value,datetime(last_modified+978307200,"unixepoch") from access order by 5 desc;'
```

↳ **[ROOT]** System TCC grants with modification times. Requires Full Disk Access for your terminal. kTCCServiceSystemPolicyAllFiles is the one that matters most.

```
sqlite3 ~/Library/Application\ Support/com.apple.TCC/TCC.db 'select
  service,client,auth_value,datetime(last_modified+978307200,"unixepoch") from access order by 4 desc;'
```

↳ User TCC grants: ScreenCapture, AppleEvents, ListenEvent, SystemPolicyDesktopFolder. AppleEvents grants are the scripting-pivot surface.

```
log show --last 7d --info --predicate 'process == "tccd" AND eventMessage CONTAINS "AUTHREQ"'
```

↳ TCC authorisation requests with the requesting client identity. Shows attempts, including denied ones, that the database alone will not.

```
tccutil reset All <bundle-id>
```

↳ Revoke every grant for a bundle. Containment action, not an investigative one -- it destroys the grant timestamps you may still need.

>> KERNEL, SYSTEM & PLUGIN LOAD POINTS

```
kmutil showloaded --list-only | grep -v com.apple
```

↳ Loaded kernel extensions excluding Apple's. On silicon with Full Security, third-party kexts cannot load at all -- any output implies Reduced Security was authorised.

```
kmutil inspect --no-header 2>/dev/null | head -30 ; sudo kmutil check
```

↳ Kext collection inventory and consistency check against the boot kext collection.

```
systemextensionsctl list
```

↳ System extensions and EndpointSecurity clients: the modern, kext-free way to get kernel-adjacent visibility -- and to take it away.

```
ls -la /Library/SecurityAgentPlugins/ /Library/ScriptingAdditions/ ~/Library/ScriptingAdditions/ 2>/dev/null
```

↳ Authorization plugins execute in the login window context; scripting additions (OSAX) inject into scriptable applications.

```
ls -la /Library/QuickLook ~/Library/QuickLook /Library/Spotlight ~/Library/Spotlight 2>/dev/null
```

↳ QuickLook and Spotlight importer plugins execute on preview and on index -- no user launch required.

```
ls -la /Library/Audio/Plug-Ins/HAL/ /Library/CoreMediaIO/Plug-Ins/DAL/ 2>/dev/null
```

↳ Audio and camera plugin paths. Loaded into privileged media daemons and a known route to bypassing TCC prompts for microphone and camera.

```
ls -la /etc/emond.d/rules/ 2>/dev/null
```

↳ Legacy emond rules. Removed in macOS 13+, still present on older estates you may inherit.

>> ACCOUNTS & REMOTE ACCESS

```
dscl . -list /Users UserShell | grep -v false
```

↳ Accounts with a real login shell. Hidden accounts with UID < 500 do not appear in the login window.

```
dscl . -read /Users/<user> AuthenticationAuthority RecordName UniqueID PrimaryGroupID NFSHomeDirectory
```

↳ Authentication authority and identity for one account. SecureToken and password-policy changes show here.

```
dscl . -read /Groups/admin GroupMembership ; dscl . -read /Groups/com.apple.access_ssh 2>/dev/null
```

↳ Admin group membership and the SSH access group. Group-based backdoors avoid touching sudoers.

```
sudo cat /etc/sudoers /etc/sudoers.d/* 2>/dev/null | grep -vE '^#|^$'
```

↳ **[ROOT]** Effective sudo rules including drop-ins. NOPASSWD entries are the finding.

```
ls -la ~/.ssh/authorized_keys /var/root/.ssh/ 2>/dev/null ; sudo cat /var/root/.ssh/authorized_keys 2>/dev/null
```

↳ Key-based backdoor access for the user and for root.

```
sudo systemsetup -getremotelogin ; sudo systemsetup -getremoteappleevents ; ls -la
  /Library/Preferences/com.apple.RemoteManagement.plist 2>/dev/null
```

↳ **[ROOT]** SSH, remote Apple Events, and ARD/remote-management state. All three are off by default.

```
sudo launchctl print-disabled system | grep -iE 'screensharing|ssh|ard|vnc'
```

↳ **[ROOT]** Whether the remote-access services are enabled at the launchd level, independent of what System Settings displays.

[05] NETWORK

Sockets, resolvers, filters and the process attribution that makes them useful.

>> CONNECTIONS & LISTENERS

```
lsof -i -P -n
```

↳ All IP sockets with the owning process. Numeric ports and addresses so nothing blocks on reverse DNS.

```
lsof -i -P -n | grep LISTEN
```

↳ Listening sockets only. On a clean endpoint this list is short and every entry is explicable.

```
lsof -nP -i -a -p <pid>
```

↳ Sockets belonging to one process. The -a makes the conditions AND rather than OR, which is the usual mistake.

```
lsof -nP -i @<ip>
```

↳ Every process communicating with a single address -- your pivot once you have one C2 candidate.

```
netstat -anv -p tcp | head -40 ; netstat -anv -p udp | head -20
```

↳ TCP and UDP tables with queue depths and owning PIDs. Queue depth reveals a stalled beacon that lsof shows as merely established.

```
nettop -P -l 1 -x
```

↳ Per-process byte counters, single sample, no interactive UI. Fast way to spot a quiet process moving real volume.

```
nettop -m tcp -P -l 1 -J bytes_in,bytes_out,rx_dupe,state
```

↳ Per-process TCP with explicit columns and remote endpoints.

```
sudo tcpdump -i pktap,all -k NP -n -w /tmp/cap.pcap
```

↳ [ROOT] Packet capture with per-packet process attribution via the pktap pseudo-interface. This is the macOS capability that has no Linux equivalent -- use it.

```
sudo tcpdump -i pktap,all -k NP -n -c 200 'not (host 17.0.0.0/8)'
```

↳ [ROOT] Attributed capture excluding Apple's address space, so the noise floor drops to something readable on a live host.

```
sudo dtrace -n 'syscall::connect*:entry { printf("%s %d", execname, pid); }' 2>/dev/null
```

↳ [ROOT] Live connect() tracing. Requires SIP disabled, so on a production silicon host prefer eslogger or pktap.

>> DNS & NAME RESOLUTION

```
scutil --dns
```

↳ Resolver configuration and search order per interface. Read the order, not just the servers -- ordering attacks are quieter than substitution.

```
sudo killall -INFO mDNSResponder ; log show --predicate 'process == "mDNSResponder"' --last 5m --info
```

↳ [ROOT] Force mDNSResponder to dump its cache and state into the unified log, then read it. The two-step is required; there is no direct cache dump any more.

```
log show --last 24h --info --predicate 'process == "mDNSResponder" AND eventMessage CONTAINS "Q_"' | head -60
```

↳ Historical query stream from the log store. Your best retrospective DNS source without a network sensor.

```
dns-sd -B _services._dns-sd._udp
```

↳ Enumerate advertised mDNS/Bonjour services. Runs until interrupted.

```
cat /etc/hosts ; ls -la /etc/resolver/ 2>/dev/null && cat /etc/resolver/*
```

↳ Static overrides and split-DNS redirection. /etc/resolver is per-domain and far less examined than /etc/hosts.

```
scutil --proxy ; networksetup -listallnetworkservices
```

↳ System proxy settings including PAC URL, and the service list to iterate over.

```
for s in $(networksetup -listallnetworkservices | tail -n +2); do echo "== $s"; networksetup -getwebproxy "$s";
  networksetup -getsecurewebproxy "$s"; networksetup -getautoproxyurl "$s"; done
```

↳ Per-service HTTP, HTTPS, and auto-proxy configuration across every interface. A PAC URL pointing anywhere unexpected is a full-traffic redirect.

>> FILTERING & ROUTING

```
sudo pfctl -s info ; sudo pfctl -s rules -vv
```

↳ [ROOT] pf enabled state, counters, and the active ruleset with per-rule hit counts. Zero counters on a rule that should be matching is informative.

```
sudo pfctl -s states | head -40 ; sudo pfctl -s nat
```

↳ **[ROOT]** Current state table and NAT/redirect rules.

```
sudo pfctl -a '*' -sr
```

↳ **[ROOT]** Rules from every anchor including dynamically injected ones. Anchors are how a rule hides from a plain pfctl -s rules.

```
cat /etc/pf.conf ; ls -la /etc/pf.anchors/
```

↳ On-disk ruleset and anchor files -- the persistent version of what pfctl reports live.

```
sudo /usr/libexec/ApplicationFirewall/socketfilterfw --getglobalstate --getblockall --getallowsigned --listapps
```

↳ **[ROOT]** Application-layer firewall: global state, block-all, signed-app allowance, and per-app inbound rules. Separate subsystem from pf and separately abusable.

```
netstat -rn | head -20 ; route -n get default
```

↳ Routing table and effective default gateway.

```
arp -an ; ndp -an 2>/dev/null
```

↳ IPv4 ARP and IPv6 neighbour caches. Duplicate MACs or an unexpected gateway MAC point at on-path activity.

```
ifconfig -a | grep -E '^[a-z]|inet ' ; networksetup -listallhardwareports
```

↳ Every interface with addresses, plus the hardware port mapping. Watch for utun, tap, bridge and awdl interfaces you cannot account for.

!! A utunN interface with no corresponding VPN configuration profile, or a new bridge/tap device, is worth a look. Note that utun0-3 and awdl0/llw0 are normal on modern macOS -- baseline your own build before calling it.

```
sudo nvram -xp | plutil -p - 2>/dev/null | grep -iA2 'current-network\|preferred-networks'
```

↳ **[ROOT]** Wi-Fi network history retained in NVRAM. Survives OS reinstall and is not covered by normal disk collection.

```
sudo defaults read /Library/Preferences/com.apple.wifi.known-networks.plist 2>/dev/null | head -40
```

↳ **[ROOT]** Known Wi-Fi networks with join timestamps and security types. Path and format have moved across releases -- verify on your build.

[06] FILESYSTEM & METADATA

Extended attributes, provenance ledgers, APFS timestamps and the anti-forensics that target them.

>> EXTENDED ATTRIBUTES

```
xattr -l <path>
```

↳ Every extended attribute with its value. First command on any suspect file.

```
ls -l@O <path>
```

↳ Attribute names alongside BSD flags (uchg, schg, hidden) in one pass, without dumping values.

```
xattr -p com.apple.quarantine <path>
```

↳ Quarantine record: flags;timestamp(hex);agent-name;event-UUID. The event UUID joins to the LaunchServices ledger below.

```
xattr -p com.apple.provenance <path> | xxd
```

↳ **[13+]** Provenance record binding a binary to the application that installed it. Introduced in macOS 13 and largely unknown to attackers.

```
xattr -p com.apple.macl <path> | xxd
```

↳ TCC file-access grant bound to a specific bundle -- how an app retains access to a file the user opened once via a picker.

```
xattr -p com.apple.metadata:kMDItemWhereFroms <path> | xxd -r -p | plutil -p -
```

↳ Full source URL chain for a download: the landing page and the direct file URL. Frequently yields the whole delivery infrastructure.

```
xattr -rl ~/Downloads /tmp /var/tmp /Users/Shared 2>/dev/null | head -60
```

↳ Recursive xattr sweep of the standard drop directories.

```
find ~ -type f -xattrname com.apple.quarantine -print 2>/dev/null
```

↳ Every quarantined file under a home directory. Faster and cleaner than the exec-per-file loop older guides use.

```
sqlite3 ~/Library/Preferences/com.apple.LaunchServices.QuarantineEventsV2 "select
  datetime(LSQuarantineTimeStamp+978307200,'unixepoch'), LSQuarantineAgentName, LSQuarantineDataURLString,
  LSQuarantineOriginURLString from LSQuarantineEvent order by 1 desc limit 50;"
```

↳ The download provenance ledger: when, by which application, from which URL, referred from where. Persists after the file is deleted.

!! A file in ~/Downloads with no quarantine xattr was either written by something that is not a quarantine-aware application -- curl, a script, an already-running process -- or had the attribute stripped. Both are findings. This is exactly why ClickFix-style curl-pipe-bash delivery evades Gatekeeper: no quarantine bit means no assessment.

```
log show --last 7d --info --predicate 'eventMessage CONTAINS "com.apple.quarantine"'
```

↳ Quarantine set and clear events from the log store, which catches strip operations the filesystem no longer records.

>> SPOTLIGHT METADATA

```
mdls <path>
```

↳ All Spotlight attributes: kMDItemDateAdded, WhereFroms, ContentType, and the download timestamps that survive touch-based timestomping.

```
mdfind "kMDItemWhereFroms == '*'" -onlyin ~
```

↳ Everything Spotlight has recorded as downloaded within a home directory.

```
mdfind "kMDItemContentType == 'public.unix-executable'" -onlyin ~ -onlyin /tmp -onlyin /Users/Shared
```

↳ Executables in user-writable space, by content type rather than by permission bit.

```
mdfind -onlyin / "kMDItemFSCreationDate >= \$time.today(-2)"
```

↳ Files created in the last two days according to the index.

```
mdutil -s / ; mdutil -sa
```

↳ Index status per volume. Indexing disabled is an anti-forensic move and it silently breaks every mdfind above.

```
ls -la /.Spotlight-V100/Store-V2/*/ 2>/dev/null
```

↳ Raw Spotlight store for offline parsing when the live index has been disabled or the host is an image.

>> TIMESTAMPS (MACB) & TIMESTOMPING

```
stat -f 'M:%Sm A:%Sa C:%Sc B:%SB %N' -t '%F %T' <path>
```

↳ mtime, atime, ctime and birthtime on one line. APFS records all four with nanosecond granularity.

```
find <dir> -type f -newerct '2026-07-01' -print0 | xargs -0 stat -f '%Sc %N' -t '%F %T'
```

↳ Inode-change-time window. ctime survives a touch of mtime, which is what most timestomping actually manipulates.

```
sudo find / -type f -mtime -1 -not -path '/System/*' -not -path '/dev/*' -not -path '/private/var/folders/*' 2>/dev/null
```

↳ **[ROOT]** Recent modifications outside the sealed system volume and the noisiest cache paths.

```
stat -f '%Sc %Sm %SB %N' -t '%s' <path> | awk '$1<$2 || $3>$2 {print "ANOMALY: " $0}'
```

↳ Flag files where ctime precedes mtime or birthtime follows mtime -- both physically impossible without deliberate manipulation.

!! Compare filesystem timestamps against kMDItemDateAdded, the quarantine ledger timestamp, and the Rosetta AOT birth time. An attacker can rewrite the first; rewriting all four consistently is rare and the divergence is your evidence.

>> FSEVENTS -- HISTORICAL FILESYSTEM CHANGE

```
sudo ls -la /.fseventsd/
```

↳ **[ROOT]** Per-volume FSEvents store. Gzipped, protobuf-adjacent logs of every create, rename, and delete. Not human-readable in place.

```
sudo cp -R /.fseventsd /tmp/fsevents-copy && sudo chown -R $(id -un) /tmp/fsevents-copy
```

↳ **[ROOT]** Copy the store out before parsing so you never write to the original.

```
for f in /tmp/fsevents-copy/0*; do gzcat "$f" 2>/dev/null | strings | grep -iE 'oah|\.helper|LaunchAgents|Downloads';
  done | sort -u
```

↳ Quick-and-dirty triage grep across FSEvents. For real work use FSEventsParser or mac_apt -- this is the on-host approximation.

```
sudo find /Volumes -maxdepth 2 -name '.fseventsd' 2>/dev/null
```

↳ **[ROOT]** FSEvents stores on attached volumes, including that DMG the user mounted once.

FSEvents is the artifact that recovers Rosetta 2 translation history after the AOT cache has been cleared: look for a folder creation under private/var/db/oah, then a <name>.aot.in_progress creation, then a rename to <name>.aot. That triplet is a first-execution record for an x86-64 binary.

>> HIDDEN CONTENT, FORKS & VOLUMES

```
ls -laO ; ls -a@
```

↳ Dotfiles, BSD flags, hidden flag, and xattr markers in the current directory.

```
sudo find / -flags +hidden -not -path '/System/*' 2>/dev/null
```

↳ **[ROOT]** Files hidden with chflags rather than a leading dot -- invisible in Finder, invisible to a plain ls, visible here.

```
ls -l <file>/..namedfork/rsrc
```

↳ Resource fork content. Non-zero size on a modern file means the fork is being used as a payload carrier.

```
find . -name '._*' -o -name '.DS_Store' | head -20
```

↳ AppleDouble carriers and Finder metadata. .DS_Store records filenames that have since been deleted from the directory.

```
hdiutil info
```

↳ Attached disk images with their backing store paths. A mounted DMG whose backing file has been deleted is still readable here.

```
mount ; diskutil list ; diskutil apfs list
```

↳ Mounted volumes, physical stores, APFS containers, and volume roles. Look for volumes with no role and no mount point.

```
log show --last 30d --predicate 'process == "diskarbitrationd"' --info | grep -iE 'mount|attach|eject'
```

↳ Historical mount, attach and eject events for images and external media. Your USB and DMG timeline.

```
plutil -p ~/Library/Preferences/com.apple.sidebarlists.plist 2>/dev/null | grep -iA3 'Name\|Alias'
```

↳ Historical list of attached volumes as Finder remembers them -- survives the volume being long gone.

```
ls -la ~/.DocumentRevisions-V100/ /.DocumentRevisions-V100/ 2>/dev/null
```

↳ Version-store copies of documents. Recovers prior content of files an attacker edited or an insider sanitised.

[07] OBFUSCATION, SPOOFING & MACH-O

Binary internals, identity spoofing, native LOLBins and encoded payload extraction.

>> MACH-O INSPECTION ON ARM64

```
file <bin> ; lipo -detailed_info <bin>
```

↳ Architecture list and per-slice offset, size, and alignment. Hidden or mismatched slices show as size anomalies.

```
lipo -thin arm64e <bin> -output /tmp/slice.arm64e
```

↳ Extract one slice for isolated analysis so your tooling is not confused by the fat header.

```
otool -arch arm64e -hv <bin>
```

↳ Mach-O header flags for the native slice: PIE, TWOLEVEL, NO_HEAP_EXECUTION, and the hardened-runtime indicators.

```
otool -arch arm64e -L <bin>
```

↳ Linked dylibs. Anything resolving through @rpath or @executable_path is hijackable if the binary lacks library validation.

```
otool -arch arm64e -l <bin> | grep -A4 LC_RPATH
```

↳ The rpath search order, which is the actual hijack surface -- the first writable directory in this list wins.

```
otool -l <bin> | grep -A5 LC_ENCRYPTION_INFO
```

↳ Encrypted text segment. cryptid 1 means packed or encrypted and your strings pass will return nothing useful.

```
otool -l <bin> | grep -A3 __RESTRICT
```

↳ A __RESTRICT segment blocks DYLD_INSERT_LIBRARIES. Its absence widens the injection surface.

```
otool -arch arm64e -l <bin> | grep -A6 LC_CODE_SIGNATURE
```

↳ Code signature load command offset and size. A zero-size or absent entry on an arm64e binary means it cannot execute natively at all.

```
nm -mu <bin> 2>/dev/null | sort -u | head -50
```

↳ Undefined symbols -- the imported API surface, which tells you capability without running anything.

```
strings -a -n 8 <bin> | grep -Ei 'https?://|/tmp/|osascript|curl|base64|keychain|\.onion'
```

↳ Embedded C2, staging paths, and interpreter calls. Note that BSD strings finds ASCII only -- install GNU strings or use a wide-char pass for UTF-16 payloads.

```
strings -a -e l <bin> | head -40 ; strings -a -e b <bin> | head -40
```

↳ Little- and big-endian 16-bit encodings, which catch strings the default ASCII pass misses.

!! disable-library-validation combined with get-task-allow on a shipping application is injection-ready by design. On arm64e, also check that pointer authentication has not been opted out of via a legacy build target.

>> NAME & IDENTITY SPOOFING

```
ls | LC_ALL=C grep '[^ -~]'
```

↳ Non-ASCII filenames: homoglyph and RTL-override candidates. BSD grep has no -P, so this is the portable form.

```
ls | perl -CS -ne 'print if /[\x{202A}-\x{202E}\x{2066}-\x{2069}]/'
```

↳ Bidirectional override characters U+202A-202E and U+2066-2069 -- the classic extension-spoofing trick that renders evil.txt.app as evil.app.txt.

```
ls | perl -CS -ne 's/([^\x00-\x7F])/sprintf"<U+%04X>",ord$1/ge; print'
```

↳ Rewrite suspicious names with every codepoint made explicit so you can see exactly what you are looking at.

```
printf '%s' "<name>" | hexdump -C
```

↳ Raw bytes of a single filename when you need to be certain.

```
plutil -p <app>/Contents/Info.plist | grep -E
  'CFBundleIdentifier|CFBundleExecutable|LSUIElement|LSBackgroundOnly|NSAppleEventsUsageDescription'
```

↳ Declared bundle identity plus the keys that hide an app from the Dock and pre-authorise Apple Events usage prompts.

```
diff <(codesign -dv <app> 2>&1 | grep -oE 'Identifier=.*' | cut -d= -f2) <(plutil -extract CFBundleIdentifier raw
  <app>/Contents/Info.plist)
```

↳ Signing identifier versus declared bundle identifier. A mismatch is deliberate; there is no accidental version of this.

```
for a in /Applications/*.app; do n=$(basename "$a" .app); which "$n" 2>/dev/null | grep -qE '^/(usr|bin|sbin)' && echo
  "SHADOWS SYSTEM BINARY: $a"; done
```

↳ Applications whose name shadows a system binary while living outside /System and /usr/bin.

>> LOLBINS -- NATIVE EXECUTION & TRANSFER

```
osascript -e '<script>' ; osascript -l JavaScript -e '<jxa>'
```

↳ AppleScript and JXA. Scripts are data, so they need no signature -- the signed osascript binary carries the trust, and its TCC grants become the attacker's.

```
osacompile -o /tmp/x.app -e '<script>'
```

↳ Compile a script into an application bundle on the fly. Instant, signed-adjacent app with no developer account required.

```
curl -sLo /tmp/p <url> && xattr -d com.apple.quarantine /tmp/p 2>/dev/null; chmod +x /tmp/p
```

↳ The canonical fetch-then-strip chain. Hunt for the xattr -d and xattr -c invocations specifically -- they are rarely legitimate.

```
sqlite3 <db> '.shell <cmd>'
```

↳ Shell escape from inside sqlite3, which is signed, present everywhere, and rarely monitored.

```
openssl enc -d -aes-256-cbc -in <in> -out <out> ; openssl base64 -d -in <in>
```

↳ Native decrypt and decode of staged payloads. No interpreter, no download.

```
security find-generic-password -w -s <service> ; security dump-keychain -d
```

↳ Credential access via the native keychain CLI. The -d form prompts per item unless the keychain is already unlocked.

```
sysadminctl -addUser <u> -password <p> -admin ; dscl . -create /Users/<u>
```

↳ Account creation and privilege change without touching the GUI.

```
screencapture -x /tmp/s.png ; sudo sysdiagnose -u -f /tmp/
```

↳ Silent screen capture and bulk host data collection -- both signed Apple binaries, both excellent exfiltration primitives.

```
tclsh ; php -r '<code>' ; perl -e '<code>' ; awk 'BEGIN{...}'
```

↳ Surviving interpreters. python and ruby were removed from the base OS in macOS 12.3, so current tradecraft leans on these and on the shell itself.

```
xattr -c -r <path>
```

↳ Mass quarantine stripping. Audit for this exact invocation in shell history and in the unified log -- there is almost no benign reason to run it recursively.

```
arch -x86_64 <bin>
```

↳ Force execution under Rosetta 2 translation. An attacker's route to the relaxed x86-64 signing regime -- and the command that populates the AOT cache in section 03.

>> ENCODED & CHAINED PAYLOAD EXTRACTION

```
plutil -convert xml1 -o - <plist> | grep -Ei 'base64|bash -c|eval|osascript|\\x|curl'
```

↳ Encoded or chained commands hidden inside job definitions. Convert first so binary plists are greppable.

```
file <plist> ; plutil -lint <plist>
```

↳ A binary plist masquerading as another type, and malformed plists that launchd will still happily load.

```
grep -aoE '[A-Za-z0-9+/]{60,}={0,2}' <file> | base64 -d 2>/dev/null | file -
```

↳ Extract long base64 blobs and identify what they decode to without writing them to disk.

```
grep -aoE '(\\x[0-9a-fA-F]{2}){8,}' <file>
```

↳ Hex-escaped payload strings, the usual second layer once base64 has been peeled.

```
log show --last 24h --info --predicate 'eventMessage MATCHES ".*[A-Za-z0-9+/]{80,}.*"'
```

↳ Long opaque blobs passing through the log stream. Noisy, but it catches staging that never touched disk.

```
sudo eslogger exec | jq -r '.event.exec.args | join(" ")' | grep -Ei 'base64|eval|xattr -c|curl.*\|.*(ba)?sh'
```

↳ **[ROOT 13+]** Live execution stream filtered to the encode/decode and curl-pipe-shell patterns. The highest-yield single detection on a live host.

[08] USER ACTIVITY, BROWSER & APP ARTIFACTS

Intent, credentials and the behavioural record. Where infostealers actually go.

Current macOS crimeware is overwhelmingly credential- and wallet-focused: it lands via malvertising or a ClickFix prompt, harvests browsers and keychains within minutes, and exfiltrates before persistence is even established. Work this section early, not last.

>> BIOME -- BEHAVIOURAL STREAMS

```
ls -la ~/Library/Biome/streams/public/ ~/Library/Biome/streams/restricted/ 2>/dev/null
```

↳ Biome is the modern user-activity framework that supplements knowledgeC. Each stream is a separate behavioural feed in SEGB format.

```
ls -la ~/Library/Biome/streams/restricted/App.MenuItem/local/
```

↳ **[26+]** New in macOS Tahoe 26: every menu selection the user makes, with timestamp and exact menu text. Reconstructs intent -- Compress, Move to Trash, Empty Trash -- not just outcome.

```
python3 ccl_segb_cli.py ~/Library/Biome/streams/restricted/App.MenuItem/local/<file> > /tmp/menuitem.txt
```

↳ **[26+]** SEGB wraps protobuf records, so it needs a parser. ccl-segb from CCL Group handles the Tahoe streams; most commercial suites did not parse App.MenuItem as of mid-2026.

```
ls -la ~/Library/Biome/streams/public/{App.Activity,App.InFocus,Notification.Usage}/local/ 2>/dev/null
```

↳ App launch, foreground focus, and notification streams. Focus duration is how you distinguish a user opening a file from a process touching it.

```
sqlite3 ~/Library/Application\ Support/Knowledge/knowledgeC.db "select
  datetime(ZOBJECT.ZSTARTDATE+978307200,'unixepoch'), ZOBJECT.ZSTREAMNAME, ZOBJECT.ZVALUESTRING from ZOBJECT where
  ZSTREAMNAME like '%App%' order by 1 desc limit 40;"
```

↳ knowledgeC application usage. Requires Full Disk Access. Still the best single source for app usage duration across a long window.

!! Biome data can be cleared by the user (rm -rf ~/Library/Biome). The system-level store holds primarily synced cross-device data rather than duplicate local execution evidence, so treat a wiped user Biome as a gap, not as something you can fill elsewhere.

>> CLIPBOARD, RECENTS & SAVED STATE

```
ls -la ~/Library/Containers/com.apple.Pasteboard*/Data 2>/dev/null ; find ~/Library -ipath '*lipboard*' -maxdepth 4
  2>/dev/null
```

↳ **[26+]** Clipboard History is new in macOS 26 and persists copied items to disk where previous releases kept only the current item in memory. Expect copied credentials and command output. Paths are still moving between point releases -- verify on your build.

```
ls -la ~/Library/Application\ Support/com.apple.sharedfilelist/ ; plutil -p ~/Library/Application\
  Support/com.apple.sharedfilelist/com.apple.LSSharedFileList.RecentDocuments.sfl* 2>/dev/null
```

↳ SharedFileLists: most-recently-used caches per application. Records paths to files that no longer exist.

```
ls -la ~/Library/Saved\ Application\ State/
```

↳ Per-app saved window state, including document paths and sometimes window contents from the last session.

```
sqlite3 "$(getconf DARWIN_USER_CACHE_DIR)/com.apple.QuickLook.thumbnailcache/index.sqlite" 'select * from files limit
  30;' 2>/dev/null
```

↳ QuickLook thumbnail cache. Recovers a visual of a file the user previewed even after deletion.

```
ls -la ~/Library/Containers/*/Data/Library/Preferences/ 2>/dev/null | head -30
```

↳ Sandboxed application containers. Application data is canonically per-user here and usually sqlite -- this is where a stealer looks first.

>> BROWSERS -- SAFARI

```
sqlite3 ~/Library/Safari/History.db "select datetime(visit_time+978307200,'unixepoch'), url, title from history_visits
  join history_items on history_items.id=history_visits.history_item order by 1 desc limit 50;"
```

↳ Safari visit history. Needs Full Disk Access. Cocoa epoch offset is 978307200 -- get this wrong and your timeline is 31 years off.

```
plutil -p ~/Library/Safari/Downloads.plist 2>/dev/null | grep -E 'DownloadEntryURL|Path|DateAdded'
```

↳ Safari download history with source URL and destination path.

```
ls -la ~/Library/Safari/Extensions/ ; plutil -p ~/Library/Safari/State.db 2>/dev/null
```

↳ Safari extensions and restored session state.

```
sqlite3 ~/Library/Safari/CloudTabs.db 'select * from cloud_tabs limit 30;' 2>/dev/null
```

↳ Tabs synced from the user's other Apple devices -- activity from hardware you do not have.

>> BROWSERS -- CHROMIUM & FIREFOX

```
sqlite3 ~/Library/Application\ Support/Google/Chrome/Default/History "select
  datetime(last_visit_time/1000000-11644473600,'unixepoch'), url, title from urls order by 1 desc limit 50;"
```

↳ Chrome history. Note the different epoch: WebKit/Chrome microseconds since 1601, not Cocoa seconds since 2001.

```
sqlite3 ~/Library/Application\ Support/Google/Chrome/Default/'Login Data' 'select origin_url, username_value,
  length(password_value) from logins;'
```

↳ Saved credential inventory. Passwords are keychain-encrypted, but the presence and count tell you the blast radius of a stealer infection.

```
ls -la ~/Library/Application\ Support/Google/Chrome/Default/Extensions/ ; for m in ~/Library/Application\
  Support/Google/Chrome/Default/Extensions/*/*/manifest.json; do echo "== $m"; python3 -c "import
  json,sys;d=json.load(open(sys.argv[1]));print(d.get('name'),d.get('permissions'))" "$m" 2>/dev/null; done
```

↳ Chrome extension inventory with declared permissions. A malicious extension needs no persistence mechanism of its own.

```
ls -d ~/Library/Application\ Support/Google/Chrome/*/Local\ Extension\ Settings/*/ 2>/dev/null
```

↳ Extension local storage, including the wallet extensions that crypto-focused stealers specifically target.

```
ls ~/Library/Application\ Support/Firefox/Profiles/ ; sqlite3 ~/Library/Application\
  Support/Firefox/Profiles/*.default*/places.sqlite "select datetime(last_visit_date/1000000,'unixepoch'), url from
  moz_places order by 1 desc limit 40;"
```

↳ Firefox history. Third epoch variant: microseconds since Unix epoch.

```
ls -la ~/Library/Application\ Support/Firefox/Profiles/*/{key4.db,logins.json,cookies.sqlite} 2>/dev/null
```

↳ Firefox credential store and cookie jar. key4.db plus logins.json is a complete, portable credential theft package.

```
find ~/Library -maxdepth 4 -ipath '*rave*' -o -ipath '*Edge*' -o -ipath '*Arc*' -o -ipath '*Opera*' 2>/dev/null | head
```

↳ Other Chromium forks. Same schema, different path -- and stealers enumerate all of them.

>> KEYCHAIN, WALLETS & CLOUD

```
ls -la ~/Library/Keychains/ /Library/Keychains/ ; security list-keychains
```

↳ Keychain inventory. The login keychain plus the SQLite keychain-2.db under the user-specific UUID directory.

```
security dump-keychain 2>&1 | grep -c 'keychain:'
```

↳ Item count without dumping secrets. Each -d invocation prompts; a stealer scripts around this with a fake password prompt via osascript.

```
log show --last 7d --info --predicate 'subsystem == "com.apple.securityd"' | grep -iE 'unlock|denied|SecKeychain'
```

↳ Keychain unlock and trust-evaluation activity. A burst of unlock attempts is the stealer signature.

```
find ~ -maxdepth 4 \( -iname '*wallet*' -o -iname '*Electrum*' -o -iname '*Exodus*' -o -iname '*Ledger*' \) 2>/dev/null
  | head -20
```

↳ Local cryptocurrency wallet stores -- the primary monetisation target in current macOS campaigns.

```
ls -la ~/Library/Mobile\ Documents/ ; brctl log --wait --shorten 2>/dev/null | head -40
```

↳ iCloud Drive container and the CloudDocs sync log. Exfiltration via the user's own iCloud leaves no outbound connection you would flag.

```
sqlite3 ~/Library/Messages/chat.db "select datetime(date/1000000000+978307200,'unixepoch'), text from message order by
  1 desc limit 30;"
```

↳ Messages database. Nanosecond Cocoa timestamps. Relevant for social-engineering delivery and for zero-click attachment cases.

```
sqlite3 ~/Library/Application\ Support/NotificationCenter/db2/db 'select * from record order by delivered_date desc
  limit 20;' 2>/dev/null
```

↳ Notification Center store. Retains notification content, including one-time codes and alerts the user dismissed.

>> SHELL & TERMINAL HISTORY

```
ls -la ~/.zsh_history ~/.bash_history ; sudo ls -la /var/root/.*history
```

↳ Shell history for the user and for root, with timestamps on the files themselves.

```
grep -nE 'curl|wget|base64|xattr|chmod \+x|osascript|sudo |ssh ' ~/.zsh_history 2>/dev/null | tail -60
```

↳ History filtered to the interesting verbs. Check HISTFILE size and whether HISTSIZE was set to zero -- an empty history is itself a finding.

```
ls -la ~/Library/Preferences/com.apple.Terminal.plist && plutil -p ~/Library/Preferences/com.apple.Terminal.plist |
  grep -iA3 'Window Settings\|CommandString'
```

↳ Terminal saved window state and startup commands.

```
last -50 ; who -a ; w
```

↳ Login history from utmpx, current sessions, and idle times.

[09] LOGS & UNIFIED LOGGING

Mechanics, predicates by category, and live telemetry that does not need SIP disabled.

>> MECHANICS

```
log show --last 1h --info --debug
```

↳ Include info and debug levels. The default omits both, which is why most people conclude the log has nothing in it.

```
log show --style ndjson --last 1h > /tmp/log.ndjson ; wc -l /tmp/log.ndjson
```

↳ Newline-delimited JSON for downstream tooling. This is the form you want for anything programmatic.

```
log show --style syslog --last 1h
```

↳ Compact one-line-per-event form for reading on a terminal.

```
sudo log collect --output /tmp/host.logarchive --last 3d
```

↳ **[ROOT]** Package the log store for offline analysis. Do this before you start experimenting on a live host, because the store is a ring buffer.

```
log show --archive /tmp/host.logarchive --predicate '<pred>' --info
```

↳ Query a collected archive with the same predicate syntax as the live store.

```
log stream --predicate '<pred>' --level debug
```

↳ Live tail. Use while you run a suspect binary in a controlled window.

```
log config --status ; log config --status --subsystem com.apple.TCC
```

↳ Current logging mode globally and per subsystem, including whether private data is redacted.

```
log stats --overview 2>/dev/null || log stats
```

↳ Log store size and per-subsystem volume. Tells you how far back the ring buffer actually reaches on this host, which is often only hours.

!! <private> redaction hides exactly the fields you want -- paths, process names, URLs. Note it in your findings and move on; do not install an unredaction profile on a live evidentiary host mid-investigation. Deploy the profile to your hunting fleet in advance instead, and it will already be there next time.

>> PREDICATES BY CATEGORY

```
--predicate 'subsystem == "com.apple.xpc.launchd"'
```

↳ Job load, unload and spawn -- persistence installation as it happens. Cross-reference section 04.

```
--predicate 'process == "launchd" AND eventMessage CONTAINS "Service"'
```

↳ Service registration events, including jobs that loaded once and were removed.

```
--predicate 'subsystem == "com.apple.syspolicy"'
```

↳ Gatekeeper assessments and notarization checks. Section 03.

```
--predicate 'process == "syspolicyd" AND eventMessage CONTAINS[c] "deny"'
```

↳ Blocked executions. Each one is something that tried to run and could not.

```
--predicate 'subsystem == "com.apple.TCC"'
```

↳ TCC prompts, grants and denials. Section 04.

```
--predicate 'process == "tccd" AND eventMessage CONTAINS "AUTHREQ"'
```

↳ Authorisation requests with the requesting client identity attached.

```
--predicate 'process CONTAINS "XProtect"'
```

↳ XProtect and XProtect Remediator scan and detection results. Section 03.

```
--predicate 'process == "oahd"'
```

↳ Rosetta 2 translation events -- x86-64 execution on silicon. Section 03.

```
--predicate 'subsystem == "com.apple.securityd"'
```

↳ Keychain and trust-evaluation activity. Section 08.

```
--predicate 'process == "mDNSResponder"'
```

↳ DNS queries and resolver state. Section 05.

```
--predicate 'subsystem == "com.apple.network"'
```

↳ Connection establishment through Network.framework, which is what modern apps actually use.

```
--predicate 'process == "kernel" AND eventMessage CONTAINS "Sandbox"'
```

↳ Sandbox violations. A process probing outside its container leaves a trail here.

```
--predicate 'eventMessage CONTAINS "com.apple.quarantine"'
```

↳ Quarantine set and clear events. Section 06.

```
--predicate 'process == "diskarbitrationd"'
```

↳ Disk image attach and volume mount history. Section 06.

```
--predicate 'eventMessage CONTAINS[c] "osascript" OR eventMessage CONTAINS[c] "curl"'
```

↳ LOLBin invocation traces. Section 07.

```
--predicate 'process == "sudo" OR eventMessage CONTAINS "TTY="'
```

↳ sudo invocations with the originating TTY, which distinguishes an interactive operator from a script.

>> LIVE SYSCALL, FILE & ENDPOINT TRACING

```
sudo eslogger exec fork exit | jq -c '[.time, .process.executable.path, .event.exec.args[0]?]'
```

↳ **[ROOT 13+]** EndpointSecurity process telemetry as JSON. No SIP changes, no kext, no DTrace. This is the correct default on Apple silicon.

```
sudo eslogger create rename unlink link | jq -c '[.time, .process.executable.path, .event]'
```

↳ **[ROOT 13+]** File lifecycle events via ES. Catches the write-then-rename pattern that both installers and droppers use.

```
sudo eslogger mount unmount | jq .
```

↳ **[ROOT 13+]** Mount and unmount events -- DMG attach in real time.

```
sudo eslogger --list-events | wc -l ; sudo eslogger --list-events | head -40
```

↳ **[ROOT 13+]** Enumerate the supported event types on this build before you write a collection script against them.

```
sudo eslogger --oslog exec
```

↳ **[ROOT 13+]** Write ES events into the unified log rather than stdout, so they persist and can be collected with log collect.

```
sudo fs_usage -w -f filesys
```

↳ **[ROOT]** All filesystem syscalls in wide format. Extremely noisy; scope it.

```
sudo fs_usage -w -f exec
```

↳ **[ROOT]** exec and posix_spawn events -- the native execsnoop.

```
sudo fs_usage -w -f pathname <proc>
```

↳ **[ROOT]** Path-level activity scoped to a single process name.

```
sudo fs_usage -w -e Terminal fs_usage
```

↳ **[ROOT]** Exclude your own tooling from the trace, which otherwise dominates the output.

```
sudo ktrace trace -S -f C4 2>/dev/null | head -40
```

↳ **[ROOT]** Kernel trace where available. Lower-level than fs_usage and correspondingly harder to read.

DTrace-based tools (opensnoop, execsnoop, dtruss) require SIP to be disabled. On Apple silicon that means authenticating a Reduced Security LocalPolicy change and rebooting -- which alters the evidence and the platform posture you documented in section 01. Use eslogger instead. There is no case where DTrace is worth that trade on a live evidentiary host.

>> AUXILIARY & LEGACY LOGS

```
cat /var/log/install.log | grep -iE 'started|success|error' | tail -60
```

↳ Package and OS install activity. Plain text, survives reboots, and is not part of the unified log ring buffer.

```
sudo praudit -l /var/audit/current | tail -40
```

↳ **[ROOT]** OpenBSM audit trail: logins, file access, kernel events. Deprecated but present, and sometimes the only surviving login record.

```
sudo ls -la /var/audit/ ; sudo auditreduce -c lo /var/audit/* 2>/dev/null | praudit -l | tail -30
```

↳ **[ROOT]** Audit log inventory and login/logout class extraction across all retained files.

```
log show --last 7d --info --predicate 'eventMessage CONTAINS "sudo"'
```

↳ sudo invocations across the whole log store, which catches what BSM missed.

```
sudo sysdiagnose -u -f /tmp/ && ls -la /tmp/sysdiagnose_*
```

↳ **[ROOT]** Bulk system state snapshot: logs, process lists, network config, spindumps. Large, slow, and comprehensive -- take one early.

[10] VOLATILE CAPTURE & EVIDENCE HANDLING

Order of volatility on a platform where memory acquisition is largely off the table.

Full physical memory acquisition on Apple silicon is not practically available: there is no reliable DMA path, no supported kernel-level acquisition driver under Full Security, and the SIP/LocalPolicy changes required by third-party approaches destroy the posture you documented in section 01. Plan for process-scoped volatile capture instead, and accept that a reboot loses what you did not take.

>> ORDER OF VOLATILITY -- WORK TOP DOWN

ACQUIRE IN THIS ORDER -- EACH LINE OUTLIVES THE ONE ABOVE IT	
1. Network state	Sockets, ARP/NDP, pf states, DNS cache. Gone on interface change.
2. Process state	Process table, argv, open handles, loaded images, env. Gone on kill.
3. Process memory	Per-process regions via vmmap/lldb. Gone on exit.
4. Log ring buffer	Unified log store. Rolls continuously; hours, not days, on busy hosts.
5. Volatile disk	/tmp, /var/folders, DARWIN_USER_TEMP_DIR. Cleared on reboot.
6. Persistent disk	Plists, databases, xattrs, FSEvents, AOT cache. Survives reboot.

>> SCOPED VOLATILE CAPTURE

```
mkdir -p /tmp/ir-$(date +%Y%m%dT%H%M%S) && cd $_ && echo "CASE=$(hostname) OP=$(id -un) UTC=$(date -u +%FT%TZ)" | tee 00-provenance.txt
```

↳ Create a timestamped collection directory and record who collected what, when, in UTC. Do this before anything else so every artifact has a container.

```
{ date -u; sw_vers; uname -a; sysctl -n machdep.cpu.brand_string hw.model; csrutil status; spctl --status; } > 01-host.txt
```

↳ Host identity and posture in one file. Build number matters -- half the commands in this guide are version-gated.

```
{ ps -axo pid,ppid,uid,lstart,etime,args; echo ---; lsof -nP -i; echo ---; netstat -anv; echo ---; arp -an; } > 02-volatile.txt 2>&1
```

↳ Process, socket, and neighbour state in a single ordered capture. Fast, and it is the snapshot you will wish you had taken.

```
sudo launchctl dumpstate > 03-launchd.txt 2>&1 ; sudo sfltool dumpbtm > 04-btm.txt 2>&1
```

↳ **[ROOT]** Complete launchd state and the background task inventory -- the two persistence sources that are painful to reconstruct after the fact.

```
sudo log collect --output 05-unified.logarchive --last 7d
```

↳ **[ROOT]** Collect the log store early. It is a ring buffer and your own investigative activity is actively pushing older events out of it.

```
sudo cp -Rp /.fseventsd 06-fseventsd && sudo find /var/db/oah -type f -exec cp -p --parents {} 07-oah/ \; 2>/dev/null
```

↳ **[ROOT]** FSEvents store and the Rosetta AOT cache, preserving modes and timestamps. Two of the highest-value artifacts on silicon.

```
sudo tar -czf 08-config.tgz /Library/Launch* ~/Library/LaunchAgents /Library/Application\ Support/com.apple.TCC /var/db/ConfigurationProfiles 2>/dev/null
```

↳ **[ROOT]** Persistence and policy configuration as a single archive.

```
sudo sysdiagnose -u -f $(pwd) 2>/dev/null
```

↳ **[ROOT]** Apple's own bulk snapshot as a backstop for everything you did not think to collect.

>> PROCESS-SCOPED MEMORY

```
vmmap -w <pid> > mem-map-<pid>.txt ; vmmap --summary <pid>
```

↳ Region map with permissions and mapped file backing. Identifies writable-executable regions, which should not exist on a hardened binary.

```
sudo lldb -p <pid> -o 'memory read --outfile /tmp/region.bin --binary <start> <end>' -o 'detach' -o 'quit'
```

↳ **[ROOT]** Dump a specific region to disk. Requires the debugger to be able to attach -- hardened runtime without get-task-allow will refuse, which is itself informative.

```
sudo lldb -p <pid> -o 'image list' -o 'thread backtrace all' -o 'detach' -o 'quit' > mem-images-<pid>.txt
```

↳ **[ROOT]** Loaded image list with UUIDs and all thread backtraces. Often sufficient without a full region dump.

```
sudo spindump <pid> 10 -file spindump-<pid>.txt -reveal
```

↳ **[ROOT]** Kernel-assisted sample with binary images and UUIDs. Attaches where lldb cannot.

```
yara -r mem.yar <pid>
```

↳ **[ROOT]** Scan live process memory with YARA by PID. See section 12 -- this is how you triage in-memory payloads without a full dump.

!! If a process refuses debugger attach and refuses YARA memory scanning, note the refusal. Hardened runtime without get-task-allow is the legitimate explanation; so is a process that has deliberately hardened itself. Both are worth recording.

>> INTEGRITY, HASHING & CHAIN OF CUSTODY

```
find . -type f -exec shasum -a 256 {} \; | sort -k2 > MANIFEST.sha256 && shasum -a 256 MANIFEST.sha256
```

↳ Hash every collected file, then hash the manifest. The manifest hash is what you record out-of-band and sign.

```
shasum -a 256 <suspect> ; md5 <suspect> ; codesign -dv --verbose=4 <suspect> 2>&1 | grep CDHash
```

↳ SHA-256 for intel sharing, MD5 for legacy feeds that still require it, and the CDHash which is what Apple's own policy databases key on.

```
xattr -w com.example.ir.case '<case-id>' <collected-file>
```

↳ Tag collected copies with a case identifier. Only ever on your copies -- never write an xattr to an original.

```
sudo hdiutil create -srcfolder . -format UDZO -o /Volumes/ext/case-$(date +%F).dmg
```

↳ **[ROOT]** Package the collection as a compressed read-only image for transfer, then hash the image itself.

```
diskutil list ; sudo diskutil unmountDisk /dev/diskN ; sudo dd if=/dev/rdiskN of=/Volumes/ext/img.dd bs=32m
  conv=noerror,sync status=progress
```

↳ **[ROOT]** Raw block image of a whole disk. On silicon with FileVault this yields encrypted blocks unless you also capture the unlock material -- confirm before you spend six hours on it.

Preferred silicon acquisition route for a full image: boot the target into Recovery, enable a Reduced-Security policy only if unavoidable, and prefer target-disk mode over a live dd. Record every posture change you make in the provenance file -- an undocumented SIP change discovered later by opposing counsel is worse than no image at all.

>> TRIAGE TOOLING WORTH HAVING

OFFLINE PARSERS -- BECAUSE ON-HOST TOOLS ONLY GO SO FAR

mac_apt	All-in-one artifact parser for live hosts and images. Broadest coverage.
macos-UnifiedLogs	Mandiant's Rust AUL parser. Reads .logarchive and raw tracev3 offline.
ccl-segb	Parses SEGB/protobuf Biome streams incl. Tahoe App.MenuItem.
FSEventsParser	Decodes /.fseventsd gzip logs into a readable timeline.
APOLLO	Correlates knowledgeC/Biome pattern-of-life across modules.
Santa	Google's binary authorisation agent. Deploy pre-incident, not during.

[11] OSQUERY

The same artifact domains, expressed as SQL you can schedule across a fleet.

```
osqueryi --line
```

↳ Start the interactive shell in line-per-column mode, which is the only readable format for wide tables like launchd and processes.

```
.schema <table>   -- then --   SELECT * FROM osquery_info;
```

↳ Inside osqueryi: column truth for your build, and the agent's own version. Schemas drift between osquery releases -- verify before trusting any query below, including these.

>> SS-02 PROCESS & EXECUTION

```
SELECT pid,name,path,cmdline,parent,uid,start_time FROM processes WHERE on_disk=0;
```

↳ Running processes whose backing file has been deleted. Classic post-exploitation cleanup that leaves the process running.

```
SELECT p.pid,p.name,p.path,s.identifier,s.authority,s.signed FROM processes p LEFT JOIN signature s ON p.path=s.path
  WHERE s.signed=0 OR s.authority IS NULL;
```

↳ Every running process whose on-disk binary fails signature validation, in one join.

```
SELECT pid,key,value FROM process_envs WHERE key LIKE 'DYLD%' OR key LIKE '%INSERT%';
```

↳ DYLD injection variables across all processes.

```
SELECT p.name,p.pid,f.path FROM process_open_files f JOIN processes p USING(pid) WHERE f.path LIKE '/tmp/%' OR f.path
  LIKE '/Users/Shared/%';
```

↳ Open handles in world-writable paths with owning process.

```
SELECT path,sha256,sha1 FROM hash WHERE path='/path/to/bin';
```

↳ Hash a specific path. Requires an exact path constraint -- there is no wildcard scan here.

```
SELECT name,bundle_identifier,path,last_opened_time FROM apps ORDER BY last_opened_time DESC LIMIT 30;
```

↳ Application inventory by last-opened time.

```
SELECT pid,path,cmdline,signing_id,team_id,parent FROM es_process_events WHERE cmdline LIKE '%base64%' OR cmdline LIKE
  '%curl%';
```

↳ EndpointSecurity process events with signing identity. Requires the ES publisher enabled in the osquery flagfile.

>> SS-04 PERSISTENCE

```
SELECT label,path,program,program_arguments,run_at_load,keep_alive FROM launchd WHERE run_at_load=1 OR keep_alive=1;
```

↳ Auto-start and self-respawning jobs, structured. Your single best fleet-wide persistence query.

```
SELECT l.label,l.path,l.program_arguments FROM launchd l WHERE l.program_arguments LIKE '%/.%' OR l.program_arguments
  LIKE '%/tmp/%';
```

↳ Jobs whose program target is a hidden dotfile or a temp path -- the current infostealer persistence shape.

```
SELECT label,key,value FROM launchd_overrides WHERE key='Disabled' AND value='1';
```

↳ Jobs disabled by override. Check for security-agent labels.

```
SELECT name,path,type,source,status FROM startup_items;
```

↳ Consolidated startup item view across mechanisms.

```
SELECT command,path,minute,hour,day_of_month FROM crontab;
```

↳ Cron entries with their schedule fields.

```
SELECT path,identifier,team,category,state FROM system_extensions;
```

↳ System extensions with team identity and enabled state.

```
SELECT name,version,linked_against FROM kernel_extensions WHERE name NOT LIKE 'com.apple%';
```

↳ Non-Apple kexts. On silicon under Full Security this should return zero rows.

```
SELECT domain,name,value,username FROM managed_policies;
```

↳ MDM-enforced policy values, which override local settings silently.

```
SELECT u.username,k.key,k.algorithm FROM users u JOIN authorized_keys k USING(uid);
```

↳ SSH authorized_keys per user with key algorithm.

```
SELECT u.username,h.command,h.history_file FROM users u JOIN shell_history h USING(uid) WHERE h.command LIKE '%curl%'
  OR h.command LIKE '%xattr%';
```

↳ Shell history across all users, filtered to the interesting verbs.

```
SELECT source,header,rule_details FROM sudoers WHERE rule_details LIKE '%NOPASSWD%';
```

↳ Passwordless sudo rules including drop-in files.

>> SS-05 NETWORK

```
SELECT l.port,l.protocol,l.address,p.name,p.path FROM listening_ports l JOIN processes p USING(pid);
```

↳ Listening ports with the owning binary path.

```
SELECT p.name,p.path,s.remote_address,s.remote_port FROM process_open_sockets s JOIN processes p USING(pid) WHERE
  s.remote_port NOT IN (0,443,80);
```

↳ Established outbound connections excluding ordinary web ports.

```
SELECT * FROM alf_exceptions; SELECT * FROM alf;
```

↳ Application-layer firewall exceptions and global configuration.

```
SELECT id,type,address FROM dns_resolvers; SELECT address,hostnames FROM etc_hosts;
```

↳ Resolver configuration and static host overrides.

```
SELECT interface,address,mask FROM interface_addresses WHERE interface LIKE 'utun%' OR interface LIKE 'tap%';
```

↳ Tunnel and tap interfaces only -- the ones that warrant an explanation.

>> SS-06 FILESYSTEM & METADATA

```
SELECT path,key,value FROM extended_attributes WHERE key='com.apple.quarantine' AND directory LIKE '/Users/%/Downloads';
```

↳ Quarantine attributes in download directories. Requires a directory constraint.

```
SELECT path,btime,ctime,mtime,mode,uid,size FROM file WHERE directory='/tmp' AND type='regular';
```

↳ Full MACB plus mode for a directory. Compare btime against mtime for timestomping.

```
SELECT path FROM mdfind WHERE query="kMDItemWhereFroms == '*'";
```

↳ Spotlight-recorded downloads via the mdfind virtual table.

```
SELECT action,path,name,vendor,checksum,time FROM disk_events;
```

↳ Disk image and removable media attach events with vendor and checksum.

```
SELECT alias,destination_id,encryption FROM time_machine_destinations;
```

↳ Backup destinations -- both an exfiltration route and a recovery source.

>> SS-01/03/07 POSTURE, SIGNING & SPOOFING

```
SELECT config_flag,enabled FROM sip_config;
```

↳ SIP configuration flags individually rather than as a single status string.

```
SELECT assessments_enabled,dev_id_enabled,version FROM gatekeeper;
```

↳ Gatekeeper assessment and Developer ID state.

```
SELECT key,value FROM plist WHERE path='/Library/LaunchDaemons/<x>.plist';
```

↳ Read an arbitrary plist as rows. Requires an exact path.

```
SELECT domain,key,value FROM preferences WHERE domain='com.apple.loginwindow';
```

↳ Preference domain contents -- login hooks and auto-login live here.

```
SELECT name,description,user_action,time FROM xprotect_reports;
```

↳ XProtect detections with the action taken and when. Proof malware was present.

```
SELECT common_name,issuer,not_valid_after,path,self_signed FROM certificates WHERE path LIKE '%System.keychain' AND
  self_signed=1;
```

↳ Self-signed roots in the system keychain -- injected CAs for TLS interception.

```
SELECT name,identifier,path,author,version FROM chrome_extensions UNION SELECT name,identifier,path,author,version FROM
  safari_extensions;
```

↳ Browser extension inventory across engines in one result set.

```
SELECT message,subsystem,category,timestamp FROM unified_log WHERE subsystem='com.apple.TCC' AND timestamp >
  (strftime('%s','now') - 86400);
```

↳ Query the unified log from osquery with a time bound. Expensive -- always bound it.

>> SCHEDULING -- FLEET DEPLOYMENT

```
osqueryd --flagfile=/etc/osquery/osquery.flags --config_path=/etc/osquery/osquery.conf
```

↳ **[ROOT]** Run the daemon with an explicit config. Enable the ES publisher in the flagfile for es_process_events to populate.

```
--disable_events=false --enable_endpointsecurity=true --enable_endpointsecurity_fim=true
```

↳ The three flags that turn on EndpointSecurity-backed tables. Without them the es_* tables return empty and you will assume the host is clean.

```
{"schedule":{"silicon_persist":{"query":"SELECT label,program,program_arguments FROM launchd WHERE
  run_at_load=1;","interval":3600,"snapshot":true}}}
```

↳ Snapshot query on an hourly interval. Use snapshot rather than differential for persistence, so you get full state each time rather than only changes.

```
osqueryi --json "SELECT * FROM launchd WHERE run_at_load=1;" | jq -S . > baseline.json ; diff <(jq -S . baseline.json)
  <(osqueryi --json '...' | jq -S .)
```

↳ Local baseline-and-diff without a server. Cheap, effective, and works on one host.

[12] YARA & DETECTION ENGINEERING

Invocation, ranked targets, arm64-aware rule bodies you can paste and run.

>> INVOCATION

```
yara -r -s -w rules.yar <path>
```

↳ Recursive scan showing matched strings with offsets, warnings suppressed.

```
yara -r -f -N rules.yar <path>
```

↳ Fast-match mode, skip symlinks. Correct choice for bulk sweeps where you want hits, not every hit.

```
yara -m -e rules.yar <path>
```

↳ Print rule metadata and namespace with each hit, so triage does not require opening the ruleset.

```
yara -t <tag> rules.yar <path>
```

↳ Run only rules carrying a tag. Tag by artifact domain and you can scan section by section.

```
yara -d filename="$(basename <f>)" -d filepath="<f>" rules.yar <f>
```

↳ Supply external variables so rules can condition on path and name as well as content.

```
yara -p 8 -r rules.yar <path>
```

↳ Eight scanner threads. Scale to core count; the performance ceiling is usually disk, not CPU.

```
yara --max-strings-per-rule=0 --stack-size=32768 rules.yar <path>
```

↳ Raise limits for large or deeply nested rulesets that otherwise refuse to compile.

```
yarac rules.yar rules.compiled && yara -C rules.compiled <path>
```

↳ Precompile once, distribute the compiled form. Faster on every host and hides rule logic from whatever is running on the endpoint.

```
sudo yara mem.yar <pid>
```

↳ **[ROOT]** Scan a live process's memory by PID.

```
for p in $(ps -axo pid=); do sudo yara -N mem.yar $p 2>/dev/null; done
```

↳ **[ROOT]** Sweep every process's memory. Slow, noisy, and occasionally the only thing that finds a fileless payload.

>> SCAN TARGETS -- RANKED BY YIELD

```
~/Downloads ~/Desktop ~/Library/Containers/*/Data/Downloads
```

↳ Initial-access landing zones. Cross-reference the quarantine ledger in section 06.

```
/tmp /var/tmp /private/var/folders/*/*/T /Users/Shared
```

↳ Staging and drop directories, including the per-user temp dirs under /var/folders that most sweeps miss.

```
~/Library/LaunchAgents /Library/LaunchAgents /Library/LaunchDaemons
```

↳ Persistence plists, and separately the binaries their ProgramArguments point at.

```
~/Library/Application\ Support /Library/Application\ Support
```

↳ Payload homes for launchd jobs -- where the plist points once you follow it.

```
/usr/local/bin /opt ~/bin ~/.local/bin
```

↳ User-writable executable paths outside SIP protection.

```
~/Library/Preferences /Library/Preferences
```

↳ Plist-based config and encoded-payload storage.

```
/Volumes/*
```

↳ Freshly attached DMGs, before their contents are copied anywhere.

```
/var/db/oah/*/*/*.aot
```

↳ **[ROOT]** Rosetta AOT files. Silicon-specific and frequently the only surviving copy of a deleted x86-64 payload. Requires root.

!! The sealed system volume is read-only and SIP-protected. Deprioritise /System and /usr/bin -- if something there is modified you have a platform-integrity problem (section 01), not a YARA problem.

>> MODULES & CONDITION ANCHORS

```
import "macho"  // macho.number_of_segments, macho.entry_point, macho.filetype, macho.flags
```

↳ Header-level triage: MH_BUNDLE, MH_DYLIB, PIE flags, and segment counts as rule conditions.

```
macho.dylibs[i].name  /  macho.rpaths[i].path  /  macho.number_of_dylibs
```

↳ Linked libraries and rpath search order as first-class rule conditions -- how you detect hijack-ready binaries structurally rather than by string.

```
import "hash"  // hash.sha256(0, filesize)  |  hash.sha256(macho.segments[i].offset, ...)
```

↳ Hash-anchored rules and section-scoped hashing, which survives appended-data padding.

```
import "math"  // math.entropy(0, filesize) > 7.2
```

↳ Entropy gate for packed or encrypted content. Tune the threshold against your own corpus; 7.2 is a starting point, not a constant.

```
uint32(0) == 0xfeedfacf  // Mach-O 64  |  0xcafebabe  // fat header (also Java class)
```

↳ Magic-number anchors. Always gate 0xcafebabe on filesize or slice count, or you will match every Java class file on the host.

```
uint32(0) == 0xcafebabe and uint32be(4) < 0x10
```

↳ Fat Mach-O rather than Java: architecture count below 16 in big-endian. This one condition eliminates the entire Java false-positive class.

>> RULE BODIES -- PASTE, TUNE, DEPLOY

```
rule ARM64E_AdHoc_In_Writable_Path {
  meta: domain="03" author="you" ref="signing posture"
  condition:
    uint32(0) == 0xfeedfacf and macho.filetype == macho.MH_EXECUTE
    and for any p in ("/tmp/","/Users/Shared/","/private/var/tmp/") : ( filepath contains p )
}
```

↳ Native Mach-O executable in a writable path. Pair with an external filepath variable via yara -d filepath=. Deliberately broad -- use it to enumerate, then narrow.

```
rule Dylib_Hijack_Surface {
  meta: domain="07"
  condition:
    macho.number_of_rpaths > 0 and
    for any i in (0..macho.number_of_rpaths-1) : (
      macho.rpaths[i].path contains "@executable_path" or
      macho.rpaths[i].path contains "/tmp"
    )
}
```

↳ Binaries whose rpath search order includes a writable or relative location -- the structural precondition for dylib planting.

```
rule Packed_High_Entropy_MachO {
  meta: domain="07"
  condition:
    uint32(0) == 0xfeedfacf and filesize > 32KB and filesize < 40MB
    and math.entropy(0, filesize) > 7.2
}
```

↳ Entropy plus size bounds plus Mach-O magic. The size window is what keeps this from matching every compressed asset on disk.

```
rule LOLBin_Staging_Chain {
  meta: domain="07"
  strings:
    $a = "xattr -c" ascii nocase
    $b = "xattr -d com.apple.quarantine" ascii nocase
    $c = /curl\s+-[a-zA-Z]*s[a-zA-Z]*\s+http/ ascii
    $d = "osascript -e" ascii
    $e = "| bash" ascii
  condition:
    2 of them
}
```

↳ The fetch-strip-execute chain in scripts, plists, and AOT files alike. Two-of-five keeps the false-positive rate workable on developer machines.

```
rule Hidden_Dotfile_Persistence_Target {
  meta: domain="04" ref="current stealer shape"
  strings:
    $p1 = "/.helper" ascii
    $p2 = "/.agent" ascii
    $p3 = "/.mainhelper" ascii
    $l1 = "ProgramArguments" ascii
    $l2 = "RunAtLoad" ascii
  condition:
    any of ($p*) and any of ($l*)
}
```

↳ A launchd plist whose program target is a hidden dotfile. Scan the LaunchAgents and LaunchDaemons directories with this and nothing else.

```
rule Rosetta_AOT_Suspicious_Strings {
  meta: domain="03" note="scan /var/db/oah as root"
  strings:
    $x = "DerivedData" ascii
    $y = /\/Users\/[a-z]{3,12}\/(Desktop|Downloads)\// ascii
  condition:
    uint32(0) == 0xfeedfacf and any of them
}
```

↳ Developer path strings surviving inside AOT files -- an attribution source when the original binary is gone.

```
rule Bidi_Override_Filename {
  meta: domain="07"
  strings:
    $rlo = { E2 80 AE }
    $lro = { E2 80 AD }
    $fsi = { E2 81 A8 }
  condition:
    any of them
}
```

↳ UTF-8 encodings of the bidirectional override codepoints. Scan directory listings and archive indexes, not just binaries.

>> DETECTION HYGIENE

```
yara -r rules.yar /Applications /System/Applications 2>/dev/null | wc -l
```

↳ Run every new rule against known-good software before deployment. A non-zero count here is your false-positive rate, measured rather than assumed.

```
for r in rules/*.yar; do echo "== $r"; yara -w "$r" /dev/null 2>&1 | head -3; done
```

↳ Compile-check every rule file in isolation so one broken rule does not silently disable a whole namespace.

```
git -C rules log --oneline -10 ; git -C rules diff HEAD~1 -- '*.yar'
```

↳ Version-control the ruleset. A detection you cannot diff is a detection you cannot defend.

[13] ATT&CK MAPPING (macOS MATRIX)

Technique to artifact domain, so coverage gaps are visible rather than assumed.

Use this as a coverage worksheet, not a checklist. Each row names the technique, the section of this guide that carries the relevant commands, and the artifact that actually proves it. If a row has no artifact you can collect on your estate, that is a visibility gap worth funding.

INITIAL ACCESS & EXECUTION	
T1566 Phishing	S06 quarantine ledger, WhereFroms xattr, S08 browser history
T1204.002 User Exec: Malicious File	S03 ExecPolicy + AOT cache, S08 Biome App.MenuItem
T1059.002 AppleScript	S07 osascript traces, S09 log predicate on osascript
T1059.004 Unix Shell	S08 shell history, S09 eslogger exec
T1553.001 Gatekeeper Bypass	S06 missing quarantine xattr, S09 syspolicy subsystem
T1553.002 Code Signing	S03 codesign adhoc/unsigned, spctl, syspolicy_check

PERSISTENCE	
T1543.001 Launch Agent	S04 ~/Library/LaunchAgents, sfltool dumpbtm
T1543.004 Launch Daemon	S04 /Library/LaunchDaemons, launchctl print system
T1547.011 Plist Modification	S04 plutil label-vs-filename mismatch loop
T1547.015 Login Items	S04 sfltool dumpbtm, legacy login items
T1546.004 Shell Config Mod	S04 /etc/zshenv, ~/.zshrc, ~/.zprofile
T1053.003 Cron	S04 crontab, /usr/lib/cron/tabs, periodic
T1098.004 SSH Authorized Keys	S04 authorized_keys for user and root
T1543.005 Container/Sys Extension	S04 systemextensionsctl, kmutil showloaded

PRIVILEGE ESCALATION & HIJACK	
T1548.004 Elevated Exec / Prompt	S04 PrivilegedHelperTools, S09 securityd log
T1078.003 Local Accounts	S04 dscl, sysadminctl, admin group membership
T1574.001 Dylib Hijacking	S07 otool -L, LC_RPATH, S12 rule body
T1574.006 DYLD_INSERT_LIBRARIES	S02 process_envs sweep, entitlement check

DEFENCE EVASION	
T1622 Debugger Evasion	S10 attach refusal, get-task-allow absence
T1497 Virtualization Evasion	S02 argv containing system_profiler, S09 exec log
T1027.002 Software Packing	S07 LC_ENCRYPTION_INFO, S12 entropy rule
T1036.005 Match Legit Name	S07 shadow-system-binary loop, bundle id mismatch
T1036.002 RTLO	S07 bidi override scan, S12 Bidi rule
T1070.004 File Deletion	S06 FSEvents, S03 AOT cache surviving the delete
T1070.006 Timestomp	S06 ctime-vs-mtime anomaly check, kMDItemDateAdded
T1562.001 Disable/Modify Tools	S01 launchctl print-disabled, S04 override keys
T1211 Rosetta Signing Bypass	S03 arch -x86_64, /var/db/oah, oahd log

CREDENTIAL ACCESS & COLLECTION	
T1555.001 Keychain	S08 security dump-keychain, securityd log burst
T1539 Steal Web Session Cookie	S08 cookies.sqlite, Chromium Cookies, Login Data
T1552.001 Creds In Files	S08 shell history, S07 strings on staged files
T1113 Screen Capture	S04 TCC ScreenCapture grant, S07 screencapture -x
T1005 Data From Local System	S06 mdfind sweeps, S08 SharedFileLists

COMMAND & CONTROL / EXFILTRATION	
T1071.001 Web Protocols	S05 lsof -i, pktap attributed capture
T1071.004 DNS	S05 mDNSResponder log, scutil --dns
T1090 Proxy	S05 scutil --proxy, per-service PAC enumeration
T1567.002 Exfil To Cloud	S08 iCloud Mobile Documents, brctl log
T1048 Exfil Over Alt Protocol	S05 nettop byte counters, netstat queue depths

[14] QUICK TRIAGE CHECKLIST

Thirty minutes on an unfamiliar Apple silicon host. Run in order.

>> PHASE 0 -- POSTURE (DO THIS FIRST OR EVERYTHING BELOW IS ADVISORY)

```
sudo bputil -d | grep -E 'Security Mode|Signed System Volume|3rd Party Kexts' ; csrutil status ; csrutil
  authenticated-root status ; spctl --status ; log config --status
```

↳ **[ROOT]** Boot policy, SIP, SSV seal, Gatekeeper, and log redaction in one line. Paste the output into your notes verbatim with a UTC timestamp.

```
sudo profiles list -all | grep -iE 'payload|identifier' ; mdutil -sa
```

↳ **[ROOT]** Configuration profiles that may be overriding policy, and Spotlight index state -- a disabled index silently breaks every mdfind in phase 2.

>> PHASE 1 -- VOLATILE CAPTURE (BEFORE YOU DISTURB ANYTHING)

```
D=/tmp/ir-$(date -u +%Y%m%dT%H%M%SZ); mkdir -p $D; { date -u; sw_vers; ps -axo pid,ppid,uid,lstart,args; lsof -nP -i;
  netstat -anv; arp -an; } > $D/volatile.txt 2>&1; echo $D
```

↳ One command, one file, everything volatile. Note the directory name it prints.

```
sudo log collect --output $D/unified.logarchive --last 7d
```

↳ **[ROOT]** The log store is a ring buffer and your own activity is pushing it. Collect now, read later.

>> PHASE 2 -- ONE COMMAND PER DOMAIN

```
ps -axo pid,ppid,user,lstart,args | grep -vE '^\s*[0-9]+\s+1 ' | less
```

↳ S02 -- process tree with true start times, launchd children filtered out.

```
find /Applications ~/Applications -maxdepth 3 -name '*.app' -exec codesign -vv {} \; 2>&1 | grep -v 'valid on disk'
```

↳ S03 -- every application failing signature validation.

```
sudo ls -laR /var/db/oah/ 2>/dev/null | head -40
```

↳ **[ROOT]** S03 -- Rosetta AOT cache. Silicon-specific execution evidence that survives payload deletion. Do not skip this one.

```
sudo sfltool dumpbtm | grep -E 'Name|Developer Name|Executable Path|Disposition'
```

↳ **[ROOT 13+]** S04 -- complete background-item inventory with developer identity.

```
ls -la@ ~/Library/LaunchAgents /Library/LaunchAgents /Library/LaunchDaemons && launchctl list | awk '$1!="-"'
```

↳ S04 -- persistence on disk versus what is actually loaded in launchd.

```
ls -la ~/.[a-z]* 2>/dev/null | grep -vE 'Trash|DS_Store|zsh|bash|ssh|config|npm|git|cache|local'
```

↳ S04 -- hidden dotfiles in the home directory. Current stealer persistence lives here.

```
lsof -i -P -n | grep -E 'LISTEN|ESTABLISHED'
```

↳ S05 -- live listeners and outbound sessions with owning process.

```
sudo pfctl -a '*' -sr 2>/dev/null; scutil --proxy; cat /etc/hosts; ls /etc/resolver/ 2>/dev/null
```

↳ **[ROOT]** S05 -- filter anchors, proxy, and resolver tampering in one pass.

```
sqlite3 ~/Library/Preferences/com.apple.LaunchServices.QuarantineEventsV2 "select
  datetime(LSQuarantineTimeStamp+978307200,'unixepoch'),LSQuarantineAgentName,LSQuarantineDataURLString from
  LSQuarantineEvent order by 1 desc limit 25;"
```

↳ S06 -- last 25 downloads with source URL and downloading agent.

```
sudo find / -type f -mtime -2 -not -path '/System/*' -not -path '/dev/*' -not -path '/private/var/folders/*'
  2>/dev/null | head -200
```

↳ **[ROOT]** S06 -- recent writes outside the sealed volume.

```
ls -R ~/Downloads /tmp 2>/dev/null | LC_ALL=C grep '[^ -~]'
```

↳ S07 -- non-ASCII filenames in drop directories.

```
otool -L <suspect> ; codesign -d --entitlements :- --xml <suspect> | plutil -p -
```

↳ S07 -- link surface and entitlement posture of your lead binary.

```
ls -la ~/Library/Biome/streams/restricted/App.MenuItem/local/ 2>/dev/null && echo 'Tahoe menu-intent artifact present
  -- collect it'
```

↳ **[26+]** S08 -- the Tahoe App.MenuItem stream. Reconstructs deliberate user action including compress-then-delete exfil patterns.

```
ls -la ~/Library/Application\ Support/*/Default/'Login Data' ~/Library/Application\
  Support/Firefox/Profiles/*/logins.json 2>/dev/null
```

↳ S08 -- browser credential stores. Presence defines the blast radius of a stealer hit.

```
log show --last 24h --info --predicate 'subsystem == "com.apple.syspolicy" OR subsystem == "com.apple.TCC" OR subsystem
  == "com.apple.xpc.launchd" OR process == "oahd"'
```

↳ S09 -- Gatekeeper, TCC, launchd, and Rosetta in a single query.

```
sudo eslogger exec | jq -r '[.time,.process.executable.path,.event.exec.args[0]?]|@tsv'
```

↳ **[ROOT 13+]** S09 -- live execution telemetry running in a second pane while you work.

```
osqueryi --line "SELECT label,program,program_arguments,run_at_load FROM launchd WHERE run_at_load=1;"
```

↳ S11 -- auto-start jobs, structured, if osquery is deployed.

```
sudo yara -r -f -w -C rules.compiled ~/Downloads /tmp /var/tmp ~/Library/LaunchAgents /Library/Launch* /var/db/oah
```

↳ **[ROOT]** S12 -- sweep every ranked target including the AOT cache in one invocation.

>> PHASE 3 -- DECIDE

FOUR OUTCOMES -- EACH HAS A DIFFERENT NEXT MOVE

Platform downgraded	SIP off, Reduced/Permissive mode, or SSV seal broken -> pull the disk.
Persistence confirmed	Plist + payload + signing anomaly agree -> contain, then scope the fleet.
Single lead, no corroboration	Widen the time window and check AOT + FSEvents + Biome.
Nothing found	Record what you could NOT see. Log ring depth, redaction, index state.

!! If SIP is off, the LocalPolicy is not Full, or profiles are enforcing policy, treat every on-host result above as advisory only and move to offline analysis of a disk image. A compromised platform can lie to every command in this guide, including the ones that report the platform state.

Closing note on negatives: a clean result from this checklist means the artifacts you checked showed nothing, on a host whose log store may only reach back hours and whose private fields are redacted by default. Write findings that state what you looked at and how far back it reached. That sentence is what makes the report defensible six months later.